



 @azdocuments

INTRODUCTION TO DBMS

SUBJECT CODE: BCS755A

MODULE-3

Name:

USN:

College:

AZ Documents

Your Engineering Study Partner

www.azdocuments.in

INTRODUCTION TO DBMS

BCS755A

MODULE-3

Relational Model: Relational Model Concepts, Relational Model Constraints and relational database schemas, Update operations, transactions, and dealing with constraint violations.

Relational Algebra: Unary and Binary relational operations, additional relational operations (aggregate, grouping, etc.) Examples of Queries in relational algebra.

Textbook 1: Ch 5.1 to 5.3, Ch 8.1 to 8.5

3.1 Introduction

The relational model represents a database as a collection of **relations**. Informally, each relation resembles a table of values, or, to some extent, a flat file of records — it is called a flat file because each record has a simple, linear structure. However, despite this resemblance, there are important differences between a relation and an ordinary file, which become clear once the formal terminology of the relational model is understood.

When a relation is viewed as a table of values, each **row** of the table represents a collection of related data values, typically corresponding to a real-world entity or relationship. The table name and the column names help interpret the meaning of the values in each row. For example, a table named STUDENT, with column names such as Name, Student_number, Class, and Major, tells us that each row represents facts about a particular student entity, and each column name specifies how to interpret the value beneath it. All values within a single column are drawn from the same data type.

In the formal terminology of the relational model, a **row is called a tuple**, a **column header is called an attribute**, and the **table itself is called a relation**. The set of permissible values for a given column is represented by a **domain**.

Domain

A **domain D** is defined as a **set of atomic values**. The term *atomic* means that, as far as the formal relational model is concerned, each value in the domain is indivisible — it cannot be broken down further into component parts. A domain is commonly specified by naming a data type from which its values are drawn, and it is also useful to give the domain a descriptive name to aid interpretation.

Some illustrative examples of domains include:

- **Usa_phone_numbers** — the set of valid ten-digit phone numbers used in the United States.
- **Social_security_numbers** — the set of valid nine-digit Social Security numbers, used as unique identifiers for individuals.
- **Names** — the set of character strings representing the names of persons.
- **Grade_point_averages** — real (floating-point) numbers between 0 and 4, representing possible GPA values.
- **Employee_ages** — integer values between 15 and 80, representing valid employee ages.
- **Academic_department_names** — the set of valid academic department names, such as Computer Science, Economics, and Physics.
- **Academic_department_codes** — the set of valid department codes, such as 'CS', 'ECON', and 'PHYS'.

These are referred to as **logical definitions of domains**. In addition to the logical definition, a **data type or format must also be specified for each domain**. For instance, Usa_phone_numbers may be defined as a character string of the form (ddd)ddd-dddd, where each *d* is a decimal digit and the first three digits represent a valid area code. Similarly, Employee_ages is defined as an integer between 15 and 80. In summary, a domain is characterized by a **name, a data type, and a format**, and additional information — such as units of measurement for a numeric domain like Person_weights — may also be included to aid interpretation.

Relation Schema

A **relation schema R**, denoted **R(A1, A2, ..., An)**, consists of a **relation name R** together with a **list of attributes A1, A2, ..., An**. Each attribute *Ai* represents the name of a *role* played by some domain *D* within the relation schema *R*; this domain *D* is called

the **domain of A_i** , and is written as **dom(A_i)**. A relation schema is thus used to *describe* a relation, with R serving as the name of that relation.

The **degree** (also called the **arity**) of a relation is defined as the **number of attributes n** present in its relation schema.

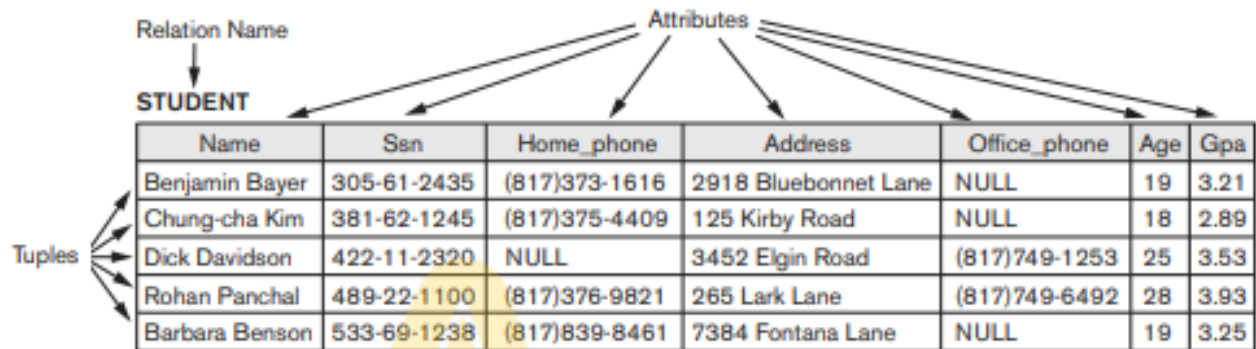


Figure 5.1
The attributes and tuples of a relation STUDENT.

Textbook Example:

A relation of degree seven, used to store information about university students, may be defined as:

STUDENT (Name, Ssn, Home_phone, Address, Office_phone, Age, Gpa)

When the data type of each attribute is also specified, the definition may instead be written as:

STUDENT (Name: string, Ssn: string, Home_phone: string, Address: string, Office_phone: string, Age: integer, Gpa: real)

Here, STUDENT is the relation name, and the relation has seven attributes. Specific domains may also be assigned to individual attributes — for example, $\text{dom}(\text{Name}) = \text{Names}$, $\text{dom}(\text{Ssn}) = \text{Social_security_numbers}$, $\text{dom}(\text{Home_phone}) = \text{USA_phone_numbers}$, $\text{dom}(\text{Office_phone}) = \text{USA_phone_numbers}$, and $\text{dom}(\text{Gpa}) = \text{Grade_point_averages}$.

It is also possible to refer to the attributes of a relation schema by their **positional order**. For instance, in the STUDENT schema above, the second attribute is Ssn, while the fourth attribute is Address.

Relation (Relation State)

A **relation** (also called a **relation state**) r of the relation schema $R(A_1, A_2, \dots, A_n)$, denoted $r(R)$, is defined as a **set of n -tuples**, written $r = \{t_1, t_2, \dots, t_m\}$. Each **n -tuple t** is an ordered list of n values, $t = \langle v_1, v_2, \dots, v_n \rangle$, where each value v_i (for $1 \leq i \leq n$) is either an element of $\text{dom}(A_i)$, or a special **NULL** value.

The i -th value in tuple t , corresponding to attribute A_i , is referred to using the notation $t[A_i]$ or $t.A_i$ (or, using positional notation, $t[i]$). The term **relation intension** refers to the schema R itself, while the term **relation extension** refers to a particular relation state $r(R)$ at a given point in time.

Formal Set-Theoretic Definition

Using set theory, a relation (or relation state) $r(R)$ can be more formally defined as a **mathematical relation of degree n** on the domains $\text{dom}(A_1), \text{dom}(A_2), \dots, \text{dom}(A_n)$, which forms a **subset of the Cartesian product** of these domains:

$$r(R) \subseteq (\text{dom}(A_1) \times \text{dom}(A_2) \times \dots \times \text{dom}(A_n))$$

The Cartesian product represents **all possible combinations** of values drawn from the underlying domains. If $|D|$ denotes the cardinality (number of values) of a domain D , then the **total number of tuples in the Cartesian product** — and hence the total number of possible tuples that could ever appear in any state of relation R — is given by:

$$|\text{dom}(A_1)| \times |\text{dom}(A_2)| \times \dots \times |\text{dom}(A_n)|$$

Out of all these theoretically possible combinations, the **current relation state** at any given time reflects only those tuples that represent **valid, real-world facts** at that moment. As the real world changes, the relation state changes correspondingly, transforming into a new relation state. In contrast, the **relation schema R is relatively static**, changing only infrequently — for example, when a new attribute is added to capture information that was not originally part of the relation.

Shared Domains and Attribute Roles

It is entirely possible for **several attributes to share the same domain**, with the attribute names indicating the different roles or interpretations of that domain within the relation. For example, in the **STUDENT** relation, the same domain **USA_phone_numbers** is used for two different attributes: **Home_phone**, representing a student's home phone number,

and Office_phone, representing a student's office phone number. A third attribute using the same domain, such as Mobile_phone, could similarly be added.

Important Notes

- A **domain** is a set of *atomic* (indivisible) values, always defined with a name, data type, and format.
- A **relation schema** is the *structure* — the relation name plus its attributes — while a **relation (state)** is the actual *set of tuples* conforming to that schema at a given time.
- The **degree (arity)** of a relation equals the number of attributes in its schema.
- Attributes can be referenced either **by name** ($t[A_i]$ or $t.A_i$) or **by position** ($t[i]$).
- A relation state is always a **subset**, not the entirety, of the Cartesian product of its attribute domains — only valid, real-world-consistent tuples are included.
- Multiple attributes in the same relation may draw from the **same domain**, with attribute names distinguishing their roles.

3.2 Characteristics of Relations

Introduction

The formal definition of a relation, discussed in the previous part, carries with it certain implications that distinguish a relation fundamentally from an ordinary file or table, even though the two may appear similar on the surface. Because a relation is defined mathematically as a *set* of tuples, and because tuples themselves are defined as ordered lists of values, several important characteristics naturally follow. These characteristics relating to the ordering of tuples, the ordering of values within a tuple, the treatment of NULL values, and the interpretation of a relation as a whole together define what makes the relational model a rigorous and abstract representation of data, rather than merely a physical storage structure.

Figure 5.2

The relation STUDENT from Figure 5.1 with a different order of tuples.

STUDENT

Name	Ssn	Home_phone	Address	Office_phone	Age	Gpa
Dick Davidson	422-11-2320	NULL	3452 Elgin Road	(817)749-1253	25	3.53
Barbara Benson	533-69-1238	(817)839-8461	7384 Fontana Lane	NULL	19	3.25
Rohan Panchal	489-22-1100	(817)376-9821	265 Lark Lane	(817)749-6492	28	3.93
Chung-cha Kim	381-62-1245	(817)375-4409	125 Kirby Road	NULL	18	2.89
Benjamin Bayer	305-61-2435	(817)373-1616	2918 Bluebonnet Lane	NULL	19	3.21

Ordering of Tuples in a Relation

A relation is formally defined as a **set of tuples**. Since, mathematically, elements of a set have **no inherent order** among them, it follows that the **tuples in a relation do not have any particular order**. In other words, a relation is *not sensitive* to the ordering of its tuples.

This is a key distinction from a file, where records are physically stored on disk or in memory, and therefore always possess a definite order allowing one to speak of the "first," "second," "i-th," or "last" record in the file. Likewise, when a relation is displayed as a table, the rows appear in some particular order on screen or on paper but this displayed ordering is not part of the relation's formal definition.

Tuple ordering is deliberately excluded from the definition of a relation because a relation is meant to represent facts at a **logical or abstract level**, independent of how those facts happen to be displayed or stored. Many different tuple orderings can be applied to the same relation for example, the tuples of a STUDENT relation could be ordered by Name, by Ssn, by Age, or by any other attribute and the relational model expresses **no preference** for any one ordering over another. Consequently, two tables showing the same tuples in different row orders are considered to represent the **same, identical relation**. Only when a relation is physically implemented as a file, or displayed as a table for viewing, does a particular ordering need to be specified.

Ordering of Values within a Tuple

According to the original definition of a relation, an n-tuple is an **ordered list** of n values, which means that the ordering of values within a tuple and correspondingly, the ordering of attributes in the relation schema is treated as significant. However, at a more abstract

conceptual level, the actual order in which attributes and their values appear is **not truly important**, as long as the correspondence between each attribute and its value is properly maintained.

Alternative Definition of a Relation

To remove the dependency on attribute ordering, an **alternative definition of a relation** can be given. In this definition:

- A relation schema $R = \{A_1, A_2, \dots, A_n\}$ is treated as a **set** of attributes (rather than an ordered list).
- A relation state $r(R)$ is a finite **set of mappings**, $r = \{t_1, t_2, \dots, t_m\}$, where each tuple t_i is a **mapping from R to D** , and D is the **union** of all the attribute domains, that is, $D = \text{dom}(A_1) \cup \text{dom}(A_2) \cup \dots \cup \text{dom}(A_n)$.
- In this definition, for every tuple t in r , it must hold that **$t[A_i]$ is in $\text{dom}(A_i)$** for each i from 1 to n .

Under this mapping-based definition, a tuple can be viewed as a **set of (attribute, value) pairs**, where each pair records the value assigned by the mapping to a particular attribute A_i . Since the attribute name travels together with its value in each pair, the **ordering of attributes becomes irrelevant** the correspondence between attribute and value no longer depends on position.

When attribute names are carried alongside their values in this way, the data is described as **self-describing data**, because the description (the attribute name) of each value is embedded directly within the tuple itself.

Note: While this alternative, order-independent definition is more general and conceptually cleaner, the **first (ordered-list) definition** of a relation is the one most commonly used in practice, because it simplifies notation. The alternative, unordered definition becomes particularly useful later, in the context of query processing and optimization.

Values and NULLs in Tuples

Each value within a tuple is required to be an **atomic value** — that is, within the framework of the basic relational model, a value is *not divisible* into smaller component parts. As a direct consequence, **composite and multivalued attributes are not permitted** in the basic relational model. Because of this restriction, the basic relational

model is sometimes referred to as the **flat relational model**, and much of the classical theory of the relational model was developed under this assumption, known as the **first normal form assumption**. Multivalued attributes must therefore be represented using separate relations, while composite attributes are represented only through their individual simple component attributes.

NULL Values

An important concept within the relational model is that of **NULL values**, which are used to represent the value of an attribute that is either **unknown** or **does not apply** to a particular tuple. A special marker, called **NULL**, is used to denote such cases.

For example, in a STUDENT relation, some tuples may show NULL for Office_phone simply because those students do not have an office — meaning the attribute does not apply to them. Another tuple might show NULL for Home_phone, either because the student genuinely has no home phone, or because the student does have one but its value is simply not known to the database.

In general, a NULL value can carry **several different meanings**, including:

- **Value unknown** — the value exists in reality but is not currently known.
- **Value exists but is not available** — the value is known to exist but has not been recorded.
- **Attribute does not apply to this tuple** — also called **value undefined** (for example, a Visa_status attribute would be undefined/inapplicable for a domestic student).

It is possible, in principle, to devise different codes to distinguish between these different meanings of NULL; however, incorporating such distinctions into relational model operations has proven difficult in practice, and this level of detail lies outside the scope of the standard presentation of the model.

Behaviour of NULLs in Comparisons

The precise meaning attached to a NULL value governs how it behaves during **arithmetic aggregations** or **comparisons** with other values. For example, comparing two NULL values with each other leads to inherent **ambiguity** — if both Customer A and Customer B have a NULL value for Address, this does **not** mean that they necessarily share the same address; it simply means that neither address is known or

applicable. Because of such ambiguities, it is considered good practice during database design to **avoid the use of NULL values as much as possible**.

Interpretation (Meaning) of a Relation

A relation schema can be interpreted in more than one way, offering different conceptual perspectives on what a relation represents.

As a Declaration or Assertion: A relation schema can be viewed as a general declaration or assertion about the type of facts the relation holds. For example, the schema of the STUDENT relation asserts that, in general, a student entity possesses a Name, Ssn, Home_phone, Address, Office_phone, Age, and Gpa. Each individual tuple within the relation can then be seen as a specific **fact**, or a particular instance confirming this general assertion — for example, one tuple might assert the fact that there exists a STUDENT named Benjamin Bayer, with a specific Ssn, a specific Age, and so on.

Entities versus Relationships: Some relations represent facts about **entities**, while other relations represent facts about **relationships** between entities. For example, a relation schema such as MAJORS(Student_ssn, Department_code) asserts that students major in particular academic disciplines — here, each tuple *relates* a student to their major department, rather than describing a standalone entity. The relational model represents both entities and relationships **uniformly**, simply as relations. While elegant, this uniform treatment can sometimes reduce understandability, since one must infer from context or naming whether a given relation represents an entity type or a relationship type.

As a Predicate: An alternative interpretation views a relation schema as a **predicate**, where the values in each tuple are the specific values that satisfy that predicate. For example, the predicate STUDENT(Name, Ssn, ...) is considered *true* precisely for those tuples that exist in the current STUDENT relation, with each tuple representing a distinct proposition or fact about the real world. This predicate-based interpretation is particularly useful in the context of logic programming languages such as Prolog, since it allows relational data to be integrated naturally within such languages.

The Closed World Assumption: Under the predicate interpretation, an assumption known as the **closed world assumption** is typically applied, which states that the **only true facts in the universe are those tuples actually present within the current extension (state) of the relation**. Any combination of attribute values that does *not* appear as a tuple in the relation is, under this assumption, considered **false**. This

interpretation becomes especially significant later, when relations are queried using relational calculus.

- Tuple ordering and value ordering within a tuple are **not part of the formal definition** of a relation — both are conveniences of display or storage, not defining properties.
- The **mapping-based (self-describing) definition** of a relation removes dependence on attribute ordering entirely, at the cost of more complex notation.
- The **flat relational model / first normal form assumption** restricts every attribute value to being atomic — no composite or multivalued attributes are allowed directly.
- NULL is a **special marker**, not an ordinary value, and it can mean "unknown," "not available," or "not applicable," depending on context.
- Comparing two NULLs does **not** imply equality of the underlying (unknown) real-world values.
- A relation may represent either an **entity type** or a **relationship type** — the relational model does not visually distinguish between the two.
- The **closed world assumption** treats absence from the relation as equivalent to falsehood.

The characteristics of relations discussed in this section — the absence of any inherent tuple ordering, the abstract irrelevance of value ordering within a tuple, the strict requirement of atomic values under the first normal form assumption, the careful handling of NULL values, and the multiple valid interpretations of a relation as an assertion, an entity/relationship representation, or a predicate — together establish the relational model as a rigorous, logically grounded framework for representing data. These properties distinguish a relation clearly from a physically ordered file structure, emphasizing that the relational model operates at a logical, set-theoretic level rather than a purely physical one.

3.3 Relational Model Concepts

Having established the core concepts of domains, attributes, tuples, and relations, along with the key characteristics that distinguish a relation from an ordinary file, it is now necessary to fix a **standard notation** for representing these concepts consistently. This

notation is used throughout the study of the relational model and relational algebra, and understanding it thoroughly makes it much easier to read, write, and interpret relational expressions, schemas, and queries correctly — particularly in later chapters dealing with relational algebra and relational calculus.

Notation for Relation Schemas and States

- A **relation schema** R of degree n is denoted by $R(A_1, A_2, \dots, A_n)$, where R is the name of the relation and A_1 through A_n are its attributes.
- Uppercase letters Q, R, S are used to denote **relation names**.
- Lowercase letters q, r, s are used to denote **relation states** — that is, the actual current set of tuples belonging to relations Q, R , and S respectively.
- The letters t, u, v are used to denote **tuples**.

Dual Meaning of a Relation Name

An important notational convention is that the **name of a relation schema**, such as **STUDENT**, is also commonly used to indicate the **current set of tuples** in that relation — that is, its current relation state. In contrast, when the relation is written explicitly with its attribute list, such as **STUDENT(Name, Ssn, ...)**, this refers **only to the relation schema** itself, and not to any particular state or set of tuples.

In other words:

- **STUDENT** → generally refers to the current relation state (the actual data).
- **STUDENT(Name, Ssn, ...)** → refers specifically to the relation schema (the structural definition).

Dot Notation for Attributes

An attribute A can be **qualified** with the name of the relation R to which it belongs, using the **dot notation** $R.A$ — for example, **STUDENT.Name** or **STUDENT.Age**. This qualification becomes necessary because the **same attribute name may be used in two different relations**; the dot notation removes any ambiguity about which relation's attribute is being referred to. It is important to note that, within a single relation, **all attribute names must be distinct** from one another.

Notation for Tuples and Their Component Values

An **n-tuple** t belonging to a relation state $r(R)$ is denoted as:

$t = \langle v_1, v_2, \dots, v_n \rangle$

where each v_i is the value corresponding to attribute A_i of the tuple.

The following notations are used to refer to the **component values** of a tuple:

- $t[A_i]$ and $t.A_i$ (and sometimes the positional form $t[i]$) both refer to the **single value** v_i in tuple t corresponding to attribute A_i .
- $t[A_u, A_w, \dots, A_z]$ and $t.(A_u, A_w, \dots, A_z)$, where $A_u, A_w, \dots, A_z$ is a list of attributes belonging to R , both refer to the **subtuple** of values $\langle v_u, v_w, \dots, v_z \rangle$ extracted from t , corresponding to just the specified list of attributes.

Textbook Example

Consider the tuple:

$t = \langle \text{'Barbara Benson'}, \text{'533-69-1238'}, \text{'(817)839-8461'}, \text{'7384 Fontana Lane'}, \text{NULL}, \text{19}, \text{3.25} \rangle$

taken from a STUDENT relation with schema STUDENT(Name, Ssn, Home_phone, Address, Office_phone, Age, Gpa). Using the notation defined above:

- $t[\text{Name}] = \langle \text{'Barbara Benson'} \rangle$ — this extracts the single value corresponding to the Name attribute.
- $t[\text{Ssn}, \text{Gpa}, \text{Age}] = \langle \text{'533-69-1238'}, \text{3.25}, \text{19} \rangle$ — this extracts the subtuple of values corresponding to the attributes Ssn, Gpa, and Age, in that specified order, regardless of their original position within the full tuple t .

Explanation: This example demonstrates that the subtuple notation $t[A_u, A_w, \dots, A_z]$ allows selective extraction of specific attribute values from a tuple, in whatever order the attributes are listed in the notation — the values returned correspond exactly to the attributes requested, not to their original positions in the full tuple.

Important Notes

- Uppercase (Q, R, S) = relation **names**; lowercase (q, r, s) = relation **states** — this distinction is used consistently in formal relational algebra notation.
- A bare relation name (e.g., STUDENT) usually means the **current state**; the same name with an attribute list (e.g., STUDENT(Name, Ssn, ...)) means the **schema**.

- Dot notation ($R.A$) is essential whenever **attribute names are shared** across multiple relations, to avoid ambiguity.
- $t[A_i]$ and $t.A_i$ are interchangeable notations for a **single attribute value** within a tuple.
- $t[A_u, A_w, \dots, A_z]$ extracts a **subtuple** — multiple values — based on a specified list of attributes.

The notation introduced in Section 5.1.3 provides a precise, standardized way to refer to relation names, relation states, tuples, and their individual or grouped attribute values. Mastery of this notation — particularly the distinction between uppercase and lowercase letters, the dual use of a relation name for both schema and state, the dot notation for resolving attribute ambiguity, and the $t[A_i]$ / $t[A_u, A_w, \dots, A_z]$ conventions for extracting values — is essential for correctly reading and writing relational algebra expressions and formal relational model definitions in the chapters that follow.

3.4 Relational Model Constraints and Relational Database Schemas

Introduction to Constraints & Domain Constraints

Introduction

Categories of Constraints

Constraints on databases can generally be divided into **three main categories**:

1. **Inherent model-based constraints (implicit constraints)** — Constraints that are built into the data model itself, simply as a consequence of how the model is defined. For example, the characteristics of relations discussed earlier — such as the rule that a relation cannot contain duplicate tuples — fall under this category, since a relation is, by definition, a *set* of tuples.
2. **Schema-based constraints (explicit constraints)** — Constraints that can be directly expressed within the schema of the data model, typically by specifying them using the **DDL (Data Definition Language)**. This is the category of constraints discussed in detail in this section — namely, domain constraints, key constraints, NULL constraints, entity integrity constraints, and referential integrity constraints.
3. **Application-based constraints (semantic constraints / business rules)** — Constraints that **cannot** be directly expressed within the schema of the data model,

and therefore must instead be expressed and enforced through **application programs**, or through other specialized mechanisms. These are more general constraints, relating to the meaning and behavior of attributes, and are typically difficult to express and enforce purely within the data model itself. In some cases, such constraints can be specified as **assertions** in SQL.

Data Dependencies — A Related Category

Another important category of constraints is **data dependencies**, which include **functional dependencies** and **multivalued dependencies**. These are used primarily for testing the "goodness" of the design of a relational database, and are central to a process called **normalization**.

Schema-Based Constraints — Overview

The **schema-based constraints**, which form the main focus of this section, include:

- **Domain constraints**
- **Key constraints**
- **Constraints on NULLs**
- **Entity integrity constraints**
- **Referential integrity constraints**

3.4.1 Domain Constraints

Domain constraints specify that, within each tuple, the **value of each attribute A must be an atomic value drawn from the domain $\text{dom}(A)$ of that attribute**. This directly builds upon the earlier definition of a domain as a set of atomic values.

The **data types** typically associated with domains include:

- Standard **numeric data types** for integers — such as short integer, integer, and long integer.
- Numeric data types for **real numbers** — such as float and double-precision float.
- **Characters, Booleans, fixed-length strings, and variable-length strings.**
- **Date, time, timestamp**, and various other special-purpose data types.

In addition to standard data types, domains can also be described using a **subrange of values** drawn from a data type (for example, integers between 15 and 80), or as an **enumerated data type**, in which all possible valid values are **explicitly listed** (for example, a domain listing only the values 'CS', 'ECON', and 'PHYS').

Important Notes

- Domain constraints are essentially an **enforcement mechanism** for the domain concept introduced earlier — every attribute value must belong to its declared domain.
- Domains may be expressed as standard data types, as subranges of a data type, or as explicitly enumerated sets of values.
- Domain constraints belong to the **schema-based (explicit)** category, since they are specified directly in the DDL when the relation schema is created.
- SQL, being the standard DDL used by most relational DBMSs, provides its own detailed set of data types for expressing domain constraints.

Database constraints exist to ensure that the data stored in a relational database remains consistent with the rules of the real-world miniworld it represents. These constraints fall into three broad categories — inherent model-based constraints, schema-based (explicit) constraints, and application-based (semantic) constraints — with the schema-based category, including domain constraints, key constraints, NULL constraints, entity integrity, and referential integrity, forming the primary focus of formal relational database design. Domain constraints, specifically, ensure that every attribute value in every tuple remains atomic and drawn strictly from its declared domain, laying the foundation for the more advanced key and integrity constraints discussed in the sections that follow.

3.4.2 Key Constraints and Constraints on NULL Values

Introduction

One of the most fundamental and frequently examined topics in relational database theory is the concept of **keys**. In the formal relational model, a relation is defined as a **set of tuples**, and since all elements of a mathematical set must be distinct by definition, it follows directly that **all tuples in a relation must also be distinct** — no two tuples can have exactly the same combination of values for all their attributes. This inherent uniqueness property forms the foundation from which the formal concepts of **superkey**,

key, candidate key, and primary key are derived, along with the related rules governing **NULL values** on key attributes.

Uniqueness and the Concept of Superkey

Because no two tuples in a relation can be entirely identical, there usually exist certain **subsets of attributes** of a relation schema R with the special property that **no two tuples in any relation state r of R can have the same combination of values** for just those attributes. If such a subset of attributes is denoted by **SK**, then for any two distinct tuples t_1 and t_2 in a relation state r of R , the following constraint must hold:

$$t_1[SK] \neq t_2[SK]$$

Any such set of attributes **SK** is called a **superkey** of the relation schema R . A superkey specifies a **uniqueness constraint**, ensuring that no two distinct tuples in any state r of R can share the same value for **SK**. It is worth noting that **every relation has at least one default superkey** — namely, the set of *all* its attributes taken together, since no two distinct tuples can agree on every single attribute value (by the very definition of a relation as a set).

From Superkey to Key

A superkey, however, **can contain redundant attributes** — attributes that are not actually necessary to guarantee uniqueness. This leads to the more useful and refined concept of a **key**, which has **no redundancy**.

A **key K** of a relation schema R is defined as a **superkey of R with the additional property that removing any attribute A from K leaves a set of attributes K' that is no longer a superkey of R** . In other words, a key satisfies **two properties**:

1. **Uniqueness** — Two distinct tuples in any state of the relation cannot have identical values for all the attributes in the key. (This uniqueness property is also shared by superkeys in general.)
2. **Minimality** — The key must be a **minimal superkey**, meaning that it is a superkey from which **no attribute can be removed** while still preserving the uniqueness constraint. This minimality property is a strict requirement for a key, but it is only **optional** for a general superkey.

From these two properties, it follows that **a key is always a superkey**, but the reverse is not necessarily true — a superkey may or may not qualify as a key, depending on whether it satisfies the minimality condition.

Textbook Example — STUDENT and Key/Superkey

Consider the STUDENT relation discussed earlier. The attribute set {Ssn} is a **key** of STUDENT, because no two student tuples can ever have the same value for Ssn. Now consider the set {Ssn, Name, Age} — since it includes Ssn, this set is automatically a **superkey** of STUDENT. However, this larger set is **not a key**, because removing Name or Age (or both) from it still leaves {Ssn}, which continues to satisfy the uniqueness property on its own — meaning the original set {Ssn, Name, Age} contained **redundant attributes** and therefore fails the minimality requirement of a key.

Important Note: In general, **any superkey formed from a single attribute is automatically also a key**, since a single attribute cannot be reduced any further. A key consisting of **multiple attributes**, however, requires **all of its attributes together** to satisfy the uniqueness property — no proper subset of it may do so alone.

Identifying Tuples Using Key Values

The value of a key attribute can be used to **uniquely identify each tuple** in a relation. For instance, the Ssn value '305-61-2435' uniquely identifies the tuple corresponding to a specific student, such as Benjamin Bayer, within the STUDENT relation.

It is important to understand that the set of attributes constituting a key is a **property of the relation schema itself** — it is a constraint that must hold for every valid state of that schema, not just the current one. A key is determined from the **meaning of the attributes** involved, and this property is **time-invariant**: it must continue to hold even as new tuples are inserted into the relation over time. For this reason, the **Name** attribute of the STUDENT relation should **not** be designated as a key, because it is entirely possible that two different students with identical names could exist at some point in a valid database state.

Candidate Keys and Primary Key

In general, a relation schema **may have more than one key**. When this is the case, each such key is referred to as a **candidate key**. For example, a CAR relation may have **two candidate keys**: License_number and Engine_serial_number — both are individually capable of uniquely identifying a tuple in the relation.

When a relation schema has multiple candidate keys, it is common practice to designate **one of them as the primary key** of the relation. The primary key is the candidate key whose values are actually used, in practice, to identify tuples within the relation. By convention, the attributes forming the **primary key** of a relation schema are shown **underlined** in schema diagrams.

The choice of which candidate key becomes the primary key is, to some extent, **arbitrary** — however, it is generally considered good practice to choose a primary key that consists of a **single attribute**, or at least a **small number of attributes**, for simplicity and efficiency. The remaining candidate keys that are not chosen as the primary key are instead designated as **unique keys**, and they are **not underlined** in schema diagrams.

Constraints on NULL Values

Apart from key constraints, another important type of constraint specifies **whether NULL values are or are not permitted** for a given attribute. For example, if every tuple of the STUDENT relation is required to have a valid, non-NULL value for the **Name** attribute, then Name is said to be constrained as **NOT NULL**.

Important Notes

- **Superkey** → a set of attributes guaranteeing uniqueness (may contain redundant attributes).
- **Key** → a *minimal* superkey (no redundant attributes; removing any attribute breaks uniqueness).
- **Every key is a superkey, but not every superkey is a key.**
- The **set of all attributes** of a relation is always a default superkey.
- A **single-attribute superkey is always a key**, automatically satisfying minimality.
- A relation may have **multiple candidate keys**; one is chosen as the **primary key** (underlined), and the rest become **unique keys**.

- Key constraints are **time-invariant** — they must hold for every valid state of the relation, not just the current one, and are chosen based on the real-world meaning of the attribute, not merely on the current absence of duplicate values.
- NULL constraints specify whether a particular attribute is permitted to have NULL values or must always be **NOT NULL**.

The concepts of superkey, key, candidate key, and primary key together define how individual tuples within a relation can be **uniquely and reliably identified**. A superkey guarantees uniqueness but may carry redundant attributes, while a key refines this by requiring minimality — no attribute can be removed without losing the uniqueness guarantee. When multiple candidate keys exist for a relation, one is conventionally chosen as the primary key, typically favoring simplicity through fewer attributes. Alongside these key constraints, NULL constraints further regulate the validity of a relation state by specifying whether particular attributes are permitted to hold NULL values, together ensuring the structural integrity of every valid relation state.

3.4.3 Relational Databases and Relational Database Schemas

Introduction

The definitions and constraints discussed so far — domain constraints, key constraints, and NULL constraints — apply primarily to **single relations** and their individual attributes. In practice, however, a **relational database** typically consists of **many relations**, with tuples across these relations related to one another in various meaningful ways. This section formally defines what constitutes a **relational database** and a **relational database schema**, using the COMPANY database as a running textbook example.

Relational Database Schema

A **relational database schema S** is formally defined as a **set of relation schemas**, together with a **set of integrity constraints**:

$$S = \{R_1, R_2, \dots, R_m\}$$

along with an associated **set of integrity constraints IC**.

Relational Database State

A **relational database state DB** of schema **S** is defined as a **set of relation states**:

$$\mathbf{DB} = \{\mathbf{r1}, \mathbf{r2}, \dots, \mathbf{rm}\}$$

such that each **ri** is a **state of Ri**, and such that all the relation states **ri**, taken together, **satisfy the integrity constraints specified in IC**.

Textbook Example — The COMPANY Database

The textbook introduces a sample relational database schema called **COMPANY**, consisting of the following six relation schemas:

COMPANY = {EMPLOYEE, DEPARTMENT, DEPT_LOCATIONS, PROJECT, WORKS_ON, DEPENDENT}

In each of these relation schemas, the **underlined attribute represents the primary key** of that relation. A corresponding **relational database state** — an actual populated snapshot of data conforming to the **COMPANY** schema — is also provided in the textbook, and this same schema and state are used throughout the following chapters for developing sample queries in various relational languages such as relational algebra, relational calculus, and SQL.

Implicit Meaning of "Relational Database"

When the term "**relational database**" is used in general discussion, it is understood to **implicitly include both its schema and its current state** together — that is, both the structural definition (schema) and the actual data (state) at a given point in time.

Valid and Invalid Database States

A **database state that does not obey all the integrity constraints** specified in **IC** is called a **not valid** (or invalid) state. Conversely, a database state that **satisfies all the constraints** in the defined set of integrity constraints **IC** is called a **valid state**. The DBMS is responsible for ensuring, as far as possible, that only valid states are allowed to exist.

Same Concept, Different Attribute Names — and Vice Versa

An important observation from the **COMPANY** schema is that the **same real-world concept** may be represented using **different attribute names** across different relations. For example, the attribute **Dnumber** in both **DEPARTMENT** and **DEPT_LOCATIONS**

refers to the same real-world concept — the number assigned to a department. However, this very same concept is referred to as **Dno** in the EMPLOYEE relation, and as **Dnum** in the PROJECT relation. Thus, attributes representing the same real-world concept **may or may not** carry identical names across different relations.

Conversely, it is also possible for **attributes representing entirely different concepts** to share the **same name** in different relations. For example, the attribute name **Name** could have been used for both the Pname attribute of PROJECT and the Dname attribute of DEPARTMENT — in which case, two attributes with the same name would actually represent two very different real-world concepts: project names and department names respectively.

Problem with Forcing Identical Names for the Same Concept

In some early versions of the relational model, an assumption was made that the same real-world concept, when represented as an attribute, should always carry an **identical attribute name** across all relations in which it appears. This assumption, however, creates problems when the **same real-world concept is used in different roles (meanings) within the very same relation**.

The textbook illustrates this with the concept of **Social Security number**, which appears **twice** within the EMPLOYEE relation itself: once representing the employee's own SSN, and once representing the SSN of that employee's supervisor. Because both roles occur within the **same relation**, they must necessarily be given **distinct attribute names** **Ssn** and **Super_ssn**, respectively — precisely in order to distinguish their different meanings, even though both conceptually represent "a Social Security number."

Data Definition Language (DDL)

Every relational DBMS must provide a **Data Definition Language (DDL)** for defining a relational database schema. Most current relational DBMSs use **SQL** for this purpose, with the SQL DDL providing the syntax needed to formally specify relation schemas, their attributes, and their associated constraints.

Important Notes

- A **relational database schema** is a *structural* concept — a set of relation schemas plus integrity constraints.
- A **relational database state** is the *actual data* — a set of relation states, one per relation schema, that together satisfy all specified integrity constraints.

- The term "**relational database**", used informally, refers to **both** the schema and the current state together.
- A database state is **valid** only if it satisfies **every** constraint in IC; otherwise, it is **not valid**.
- The **same real-world concept** may have **different attribute names** in different relations (e.g., Dnumber / Dno / Dnum, all referring to department number).
- **Different concepts** may coincidentally share the **same attribute name** across relations (e.g., a generic "Name" used for both project and department names).
- When the **same concept plays two different roles within a single relation** (e.g., SSN of employee vs. SSN of supervisor, both within EMPLOYEE), **distinct attribute names must be used** (Ssn vs. Super_ssn) to avoid ambiguity.
- SQL serves as the standard **DDL** used by most current relational DBMSs to define relational database schemas.

A relational database extends the concept of a single relation to encompass a **collection of related relation schemas**, together forming a relational database schema, and a corresponding collection of relation states, forming a relational database state — with validity determined by whether all specified integrity constraints are satisfied. The COMPANY database, comprising EMPLOYEE, DEPARTMENT, DEPT_LOCATIONS, PROJECT, WORKS_ON, and DEPENDENT, illustrates how real-world concepts such as department numbers may be named differently across relations, while the same relation may require distinct attribute names to represent different roles of an otherwise identical concept, such as SSN in EMPLOYEE. These naming considerations, together with the formal notions of schema and state, set the stage for the entity integrity and referential integrity constraints discussed next.

3.4.4 Entity Integrity, Referential Integrity, and Foreign Keys

Beyond domain constraints, key constraints, and NULL constraints, the relational model recognizes **two additional, highly important types of constraints** that are considered a core part of the model itself: **entity integrity** and **referential integrity**. While key constraints and entity integrity are specified on **individual relations**, referential integrity is unique in that it is specified **between two relations**, and is used to maintain **consistency among tuples across those relations**. Together, these constraints ensure that a relational database not only avoids duplicate or ambiguous tuples, but also

maintains logically consistent relationships between data stored across multiple tables — a concept central to how relational databases model real-world associations.

Entity Integrity Constraint

The **entity integrity constraint** states that **no primary key value can be NULL**. This restriction exists precisely because the **primary key value is used to identify individual tuples** within a relation. If NULL values were permitted for the primary key, it would imply that we **cannot properly identify** some tuples in the relation. For example, if two or more tuples were allowed to have NULL for their primary key, it would become impossible to distinguish between them — which becomes especially problematic if other relations attempt to **reference** those tuples.

Important Note: Key constraints and entity integrity constraints are both specified on **individual relations** — they do not involve any relationship between two different relations.

Referential Integrity Constraint

Unlike entity integrity, the **referential integrity constraint** is specified **between two relations**, and its purpose is to **maintain consistency among tuples** in those two relations. Informally, the referential integrity constraint states that **a tuple in one relation that refers to another relation must refer to an existing tuple in that relation** — in other words, references between relations must always point to something that actually exists.

Textbook Example: In the COMPANY database, the attribute **Dno** of the EMPLOYEE relation gives the department number for which each employee works. Referential integrity requires that the value of Dno in every EMPLOYEE tuple must **match the Dnumber value of some existing tuple** in the DEPARTMENT relation — an employee cannot be recorded as working for a department that does not exist in the database.

Formal Definition of Foreign Key

To define referential integrity more precisely, the textbook first formally defines the concept of a **foreign key**. A set of attributes **FK** in a relation schema **R1** is said to be a **foreign key of R1 that references relation R2** if it satisfies the following two conditions:

1. **Domain compatibility:** The attributes in FK have the **same domain(s)** as the primary key attributes **PK** of R2. In this case, the attributes FK are said to **reference** or **refer to** the relation R2.
2. **Referencing rule:** A value of FK in a tuple **t1** of the current state $r1(R1)$ must either:
 - occur as a value of PK for **some tuple t2** in the current state $r2(R2)$ — in which case $t1[FK] = t2[PK]$, and we say that tuple **t1 references** or **refers to** tuple **t2**; or
 - be **NULL**.

In this definition, **R1** is called the **referencing relation**, and **R2** is called the **referenced relation**. When both of these conditions hold, a **referential integrity constraint from R1 to R2** is said to exist. In a typical database containing many relations, there are usually **many referential integrity constraints** operating simultaneously.

Understanding the Role or Meaning of Attributes

To properly specify referential integrity constraints, it is essential to first have a **clear understanding of the meaning or role** that each attribute (or set of attributes) plays within the various relation schemas of the database. Referential integrity constraints typically **arise naturally from the relationships** among the real-world entities that the relation schemas represent.

Textbook Example — EMPLOYEE referencing DEPARTMENT:

In the EMPLOYEE relation, the attribute **Dno** refers to the department for which an employee works. We therefore designate **Dno as a foreign key of EMPLOYEE, referencing the DEPARTMENT relation**. This means that the value of Dno in any tuple **t1** of EMPLOYEE must match the value of the **primary key of DEPARTMENT** — the **Dnumber** attribute — in some tuple **t2** of DEPARTMENT, or the value of Dno may be **NULL**, if the employee does not currently belong to any department, or will be assigned to one at a later time. For example, in the COMPANY database state, the tuple for employee 'John Smith' references the tuple for the 'Research' department, indicating that John Smith works for that department.

Foreign Key Referencing Its Own Relation

An important and often-tested observation is that a **foreign key can refer to its own relation** that is, a relation can reference itself. In the COMPANY database, the attribute **Super_ssn** in EMPLOYEE refers to the **supervisor** of an employee — and since a supervisor is also simply another employee, this reference points to **another tuple within the same EMPLOYEE relation**. Hence, **Super_ssn is a foreign key that references the EMPLOYEE relation itself**. For example, the tuple for employee 'John Smith' references the tuple for employee 'Franklin Wong', indicating that Franklin Wong is the supervisor of John Smith.

Diagrammatic Representation

Referential integrity constraints can be displayed **diagrammatically** by drawing a **directed arc (arrow)** from each foreign key to the relation it references. For clarity, the arrowhead is typically drawn **pointing** directly to the **primary key** of the referenced relation. This kind of diagram, applied to the COMPANY schema, visually shows all the referential integrity constraints existing across the six relations of the database.

Enforcement via DDL

All integrity constraints — including referential integrity — should ideally be **specified as part of the relational database schema definition itself**, if the DBMS is expected to **automatically enforce** these constraints on every database state. For this reason, the DDL includes specific provisions for defining the various types of constraints. In SQL, the **CREATE TABLE** statement of the SQL DDL allows the definition of **primary key, unique key, NOT NULL, entity integrity, and referential integrity constraints**, among various other types of constraints.

Important Notes

- **Entity integrity** → No primary key value can be NULL; applies to a **single relation**.
- **Referential integrity** → Values of a foreign key must either match an existing primary key value in the referenced relation, or be NULL; applies **between two relations**.
- A **foreign key (FK)** in R1 referencing R2 must (1) share the same domain as the primary key (PK) of R2, and (2) for every tuple, either match some PK value in R2 or be NULL.
- **R1 = referencing relation, R2 = referenced relation.**

- A foreign key **can reference its own relation** — a recursive relationship, as seen with Super_ssn in EMPLOYEE.
- Referential integrity constraints are typically shown diagrammatically using a **directed arc**, with the arrowhead pointing to the primary key of the referenced relation.
- SQL's **CREATE TABLE** statement provides the DDL syntax for specifying entity integrity and referential integrity constraints.

Entity integrity and referential integrity together form two of the most fundamental structural rules of the relational model, ensuring that data remains both individually identifiable and consistently interrelated across a database. Entity integrity guarantees that every tuple in a relation can be uniquely identified by disallowing NULL primary key values, while referential integrity, built on the formal concept of a foreign key, ensures that references from one relation to another always point to genuinely existing tuples, or are explicitly marked as NULL. The EMPLOYEE relation's Dno and Super_ssn attributes illustrate both a standard foreign key reference to another relation (DEPARTMENT) and a self-referencing foreign key within the same relation, showcasing the flexibility and expressive power of referential integrity in modeling real-world relationships.

3.4.5 Other Types of Constraints

Introduction

Beyond the domain, key, NULL, entity integrity, and referential integrity constraints — all of which are formally included in the Data Definition Language because they occur so commonly across most database applications — there exists a broader class of constraints that fall **outside** the scope of the DDL. These are known as **semantic integrity constraints**, and because they cannot be directly expressed within the schema of the relational model, they must instead be **specified and enforced through other means**, typically within application programs. This final part also introduces the distinction between **state constraints** and **transition constraints**, completing the overall picture of constraint types in the relational model.

Semantic Integrity Constraints

Semantic integrity constraints form a class of **general constraints** that are **not part of the DDL**, and therefore must be specified and enforced in a different way from domain,

key, and integrity constraints. These constraints typically relate to the deeper **meaning and behavior** of attributes, often reflecting business rules that go beyond simple structural rules.

Textbook Examples of Semantic Constraints:

- *"The salary of an employee should not exceed the salary of the employee's supervisor."*
- *"The maximum number of hours an employee can work on all projects per week is 56."*

These types of constraints are considerably more complex and specific to the meaning of the data than domain or key constraints, and they generally **cannot** be enforced simply by declaring a data type, a key, or a NULL restriction.

Enforcement of Semantic Constraints

Such semantic constraints can be specified and enforced in a few different ways:

- Within the **application programs** that perform updates to the database — this is the **most common** approach in practice.
- Using a **general-purpose constraint specification language**.
- Through mechanisms called **triggers and assertions** in SQL, specifically using the **CREATE ASSERTION** and **CREATE TRIGGER** statements, which allow some semantic constraints to be specified declaratively within the database itself.

Important Note: It is generally **more common** to check for semantic constraints **within application programs**, rather than relying on constraint specification languages, since such languages are often **difficult and complex to use** in practice.

State Constraints vs. Transition Constraints

The constraint types discussed throughout this section — domain, key, NULL, entity integrity, referential integrity, and semantic constraints — may collectively be referred to as **state constraints**, because they define the conditions that **any single valid state** of the database must satisfy, independent of how that state was reached.

In contrast, another type of constraint, called a **transition constraint**, can be defined to deal specifically with **state changes** in the database — that is, rules governing how the

database is **permitted to move from one state to another**, rather than rules about any one state in isolation.

Textbook Example of a Transition Constraint: *"The salary of an employee can only increase."*

This is a transition constraint rather than a state constraint because it does not describe what a single valid state must look like on its own — instead, it describes a **restriction on the relationship between two consecutive states** (the state before an update, and the state after it), specifically that a salary value observed after an update must never be **less than** its value before the update.

Transition constraints are typically enforced by **application programs**, or specified using **active rules and triggers**, similar to how many semantic constraints are handled.

Important Notes

- **Semantic integrity constraints** are constraints that **cannot** be expressed directly in the DDL/schema, unlike domain, key, and integrity constraints.
- Semantic constraints are usually enforced within **application programs**, though SQL's **CREATE ASSERTION** and **CREATE TRIGGER** statements can express some of them declaratively.
- Application programs are generally **preferred** over constraint specification languages for enforcing semantic constraints, due to the complexity of such languages.
- **State constraints** define what makes **any single database state** valid, on its own.
- **Transition constraints** define what changes are **permitted between one state and the next** — they compare a "before" state to an "after" state.
- A transition constraint example: *"salary can only increase"* — this cannot be checked by looking at a single state alone; it requires comparing two states across a change.

Semantic integrity constraints and transition constraints extend the discussion of relational database constraints beyond what can be captured purely within the schema and its DDL. While domain, key, NULL, entity integrity, and referential integrity constraints can all be enforced automatically by the DBMS through schema definitions, semantic constraints — reflecting deeper business rules such as salary limits relative to

a supervisor, or maximum weekly working hours — generally require enforcement through application programs, or through SQL mechanisms such as assertions and triggers. Transition constraints further extend this picture by governing not just the validity of a single database state, but the legitimacy of the **changes** made between successive states, such as ensuring that an employee's salary is never permitted to decrease. Together, these constraint types complete the full spectrum of rules that govern the correctness and reliability of a relational database.

3.5 – Update Operations, Transactions, and Dealing with Constraint Violations

Introduction

The operations that can be performed on a relational database can broadly be categorized into two types: **retrievals** and **updates**. Retrieval operations are specified using **relational algebra** (a procedural query language, discussed in detail in Chapter 8) and **relational calculus** (a declarative query language, which defines a query without specifying the exact order of operations). A relational algebra expression forms a **new relation** by applying a sequence of algebraic operators to an existing set of relations, and this new relation becomes the **answer, or result, of the user's query**.

This section, however, focuses specifically on the **database modification, or update, operations**. There are **three basic operations** that can change the state of relations within a database:

- **Insert** — used to insert one or more new tuples into a relation.
- **Delete** — used to delete existing tuples from a relation.
- **Update (or Modify)** — used to change the values of some attributes in existing tuples.

Whenever any of these three operations is applied, the **integrity constraints specified on the relational database schema must not be violated**. This section discusses, for each type of operation, which constraints may potentially be violated, and what actions the DBMS may take if a violation occurs — using the COMPANY database as the running example, and considering **domain constraints, key constraints, entity integrity constraints, and referential integrity constraints**.

3.5.1 The Insert Operation

The **Insert operation** provides a list of attribute values for a new tuple **t** that is to be inserted into a relation **R**. Insert has the potential to violate **any of the four types of constraints** discussed in this section:

- **Domain constraints** can be violated if an attribute value is provided that **does not belong to the corresponding domain**, or is not of the appropriate data type.
- **Key constraints** can be violated if a key value in the new tuple **t** **already exists** in another tuple of the relation **r(R)**.
- **Entity integrity** can be violated if **any part of the primary key** of the new tuple **t** is **NULL**.
- **Referential integrity** can be violated if the value of **any foreign key** in **t** **refers to a tuple that does not exist** in the referenced relation.

Textbook Examples of Insert Operations

Example 1 — Entity Integrity Violation:

Insert <'Cecilia', 'F', 'Kolonsky', **NULL**, '1960-04-05', '6357 Windy Lane, Katy, TX', F, 28000, **NULL**, 4> into EMPLOYEE.

Result: This insertion **violates the entity integrity constraint**, since the value provided for the primary key attribute **Ssn** is **NULL**. The insertion is therefore **rejected**.

Example 2 — Key Constraint Violation:

Insert <'Alicia', 'J', 'Zelaya', '999887777', '1960-04-05', '6357 Windy Lane, Katy, TX', F, 28000, '987654321', 4> into EMPLOYEE.

Result: This insertion **violates the key constraint**, because another tuple with the **same Ssn value** ('999887777') **already exists** in the EMPLOYEE relation. The insertion is therefore **rejected**.

Example 3 — Referential Integrity Violation:

Insert <'Cecilia', 'F', 'Kolonsky', '677678989', '1960-04-05', '6357 Windswept, Katy, TX', F, 28000, '987654321', 7> into EMPLOYEE.

Result: This insertion **violates the referential integrity constraint** specified on the Dno attribute of EMPLOYEE, because **no corresponding tuple exists in DEPARTMENT** with Dnumber = 7. The insertion is therefore **rejected**.

Example 4 — Valid Insertion:

Insert <'Cecilia', 'F', 'Kolonsky', '677678989', '1960-04-05', '6357 Windy Lane, Katy, TX', F, 28000, NULL, 4> into EMPLOYEE.

Result: This insertion **satisfies all constraints**, and is therefore **acceptable**.

Handling Insert Violations

If an insertion violates one or more constraints, the **default option is to reject the insertion**. In such a case, it is useful if the DBMS can inform the user of the specific **reason** the insertion was rejected. A second possible approach is to **attempt to correct** the cause of the rejection — though this correction approach is **not typically used for Insert violations**; it is far more commonly applied to violations caused by **Delete and Update** operations instead.

How corrections might work for Insert (illustrative):

- In **Example 1** (NULL primary key), the DBMS could **ask the user to provide a valid Ssn value**, and then accept the insertion once a valid value is supplied.
- In **Example 3** (invalid Dno = 7), the DBMS could either **ask the user to change Dno** to a valid, existing department number (or set it to NULL), **or** it could ask the user to **first insert a DEPARTMENT tuple with Dnumber = 7**, and only accept the original EMPLOYEE insertion after that DEPARTMENT insertion succeeds.

Important Note — Cascading Violations: In the latter case, it is possible for the **insertion violation to cascade back**. For instance, if the user attempts to insert a new DEPARTMENT tuple for department 7, but provides a value for **Mgr_ssn** that does not itself exist in the EMPLOYEE relation, this new insertion would in turn cause **another referential integrity violation**, requiring further correction before the original insertion could succeed.

- Insert can potentially violate **all four** constraint types: domain, key, entity integrity, and referential integrity.
- **Domain violation** → wrong data type or value outside the domain.
- **Key violation** → duplicate key value already present in the relation.
- **Entity integrity violation** → NULL value in (any part of) the primary key.

- **Referential integrity violation** → foreign key value does not match any existing primary key value in the referenced relation (and is not NULL).
- The **default response** to any Insert violation is **rejection**, ideally accompanied by a reason.
- Correction (rather than outright rejection) is **more commonly used for Delete and Update**, not Insert — though it is still conceptually possible for Insert.
- Correcting one violation (e.g., inserting a missing DEPARTMENT tuple) can itself **cascade** into new violations if the correcting data is also invalid.

The Insert operation, while conceptually simple adding a new tuple to a relation must be checked against **all four fundamental constraint types**: domain, key, entity integrity, and referential integrity. A violation of any of these results, by default, in the **rejection** of the insertion, though the DBMS may optionally assist the user in correcting the underlying issue, such as supplying a missing primary key value or first inserting a referenced tuple that does not yet exist. This illustrates how constraint enforcement is not merely a theoretical property of the relational model, but an **active, operational safeguard** applied every time the database is modified.

3.5.2 The Delete Operation

Introduction

Unlike the Insert operation, which can potentially violate all four constraint types, the **Delete operation** has a much narrower scope of risk. Deleting a tuple can **only violate referential integrity** since removing a tuple does not, by itself, introduce any domain, key, or entity integrity issues within the relation it is deleted from. However, referential integrity violations caused by Delete can be significant, especially when the tuple being deleted is heavily referenced by tuples across multiple other relations, as demonstrated in the textbook examples below.

When Does Delete Violate a Constraint?

The Delete operation can violate **only referential integrity**. This occurs specifically when the tuple being deleted is **referenced by foreign keys from other tuples** in the database. To specify a deletion, a **condition on the attributes** of the relation is used to select the tuple (or tuples) to be deleted.

Textbook Examples of Delete Operations

Example 1 — Acceptable Deletion:

Delete the WORKS_ON tuple with **Essn** = '999887777' and **Pno** = 10.

Result: This deletion is **acceptable**, and it deletes exactly **one tuple**. Since WORKS_ON tuples are not referenced by any other relation as a foreign key target, no referential integrity violation occurs.

Example 2 — Unacceptable Deletion (Single Reference):

Delete the EMPLOYEE tuple with **Ssn** = '999887777'.

Result: This deletion is **not acceptable**, because there are **tuples in WORKS_ON that refer to this tuple** (via the **Essn** foreign key). If the EMPLOYEE tuple were deleted, those WORKS_ON tuples would be left referencing a non-existent employee, resulting in a **referential integrity violation**.

Example 3 — Unacceptable Deletion (Multiple References):

Delete the EMPLOYEE tuple with **Ssn** = '333445555'.

Result: This deletion **causes even worse referential integrity violations**, because this particular tuple is referenced by tuples from **four different relations**: EMPLOYEE (via Super_ssn — this employee supervises others), DEPARTMENT (via Mgr_ssn — this employee manages a department), WORKS_ON (via Essn — this employee works on projects), and DEPENDENT (via Essn — this employee has dependents recorded).

Options for Handling Delete Violations

When a deletion operation would cause a referential integrity violation, the DBMS has **several available options**:

- 1. Restrict:** The first option, called **restrict**, is simply to **reject the deletion** outright. This is the safest and most conservative option, ensuring that no dangling references are ever created.
- 2. Cascade:** The second option, called **cascade**, is to **attempt to propagate the deletion** by also deleting the tuples that reference the tuple being deleted. For example, in Example 2 above, the DBMS could **automatically delete** the offending WORKS_ON tuples with **Essn** = '999887777', so that no dangling reference remains after the EMPLOYEE tuple itself is removed.

3. Set Null or Set Default: The third option, called **set null** (or **set default**), is to **modify the referencing attribute values** that would otherwise cause the violation. Each such offending value is either changed to **NULL**, or changed to reference some other **default valid tuple** instead.

Important Restriction on Set Null: If the referencing attribute that causes a violation is itself **part of a primary key**, it **cannot be set to NULL** — doing so would violate the **entity integrity constraint** instead, simply trading one violation for another.

Combining Options — Textbook Example (Ssn = '333445555')

To handle Example 3 (deleting the **EMPLOYEE** tuple with Ssn = '333445555') without causing a violation, the DBMS might use a **combination of the three options**:

- **Cascade:** Automatically **delete all tuples** from **WORKS_ON** and **DEPENDENT** that have Essn = '333445555', since these tuples exist specifically to describe this employee's work assignments and dependents, and make little sense without the employee's record.
- **Set Null:** For tuples in **EMPLOYEE** with Super_ssn = '333445555' (employees supervised by this person), and the tuple in **DEPARTMENT** with Mgr_ssn = '333445555' (the department this person manages), the Super_ssn and Mgr_ssn values could **instead be changed to other valid values, or set to NULL**.

Important Note: While it may make good sense to **automatically cascade-delete** the **WORKS_ON** and **DEPENDENT** tuples referencing this employee, it generally **does not make sense** to automatically delete **other EMPLOYEE tuples** (the people this person supervised) or the **DEPARTMENT tuple** (the department this person managed) — deleting an employee should not automatically delete their subordinates or their entire department.

DBMS Support for Specifying Options

In general, when a **referential integrity constraint is specified in the DDL**, the DBMS allows the **database designer to specify which of these options** (restrict, cascade, or set null/set default) should apply in the event of a violation of that particular constraint. This means the designer can choose different handling strategies for different foreign key relationships, depending on what makes logical sense for each one.

Important Notes

- Delete can violate **only referential integrity** — never domain, key, or entity integrity.
- A violation occurs only when the tuple being deleted is **referenced by a foreign key** from some other tuple.
- **Three handling options** exist: **Restrict** (reject deletion), **Cascade** (delete referencing tuples too), and **Set Null/Set Default** (change the referencing value).
- **Set Null cannot be applied** if the referencing attribute is part of a **primary key**, since that would violate entity integrity instead.
- Different foreign keys in the same schema may reasonably use **different options** — e.g., cascade for WORKS_ON/DEPENDENT, but set-null for Super_ssn/Mgr_ssn.
- The choice of option for **each** referential integrity constraint is typically specified by the **database designer** in the **DDL**.

The Delete operation, while structurally simpler than Insert in that it can only ever threaten **referential integrity**, requires careful handling precisely because a single deleted tuple may be referenced from **multiple other relations simultaneously**, as seen with the EMPLOYEE tuple for Ssn = '333445555'. The three available strategies — restrict, cascade, and set null/set default — give database designers the flexibility to choose the most logically appropriate response for each individual referential integrity constraint, recognizing that a uniform, one-size-fits-all policy (such as always cascading) would often produce unintended and undesirable side effects, such as deleting an employee's entire department along with their own record.

3.5.3 – The Update Operation

Introduction

The **Update (or Modify)** operation is the third and final basic operation that can change the state of a relational database. Unlike Insert, which introduces an entirely new tuple, or Delete, which removes an existing tuple, Update **changes the values of one or more attributes** within a tuple (or tuples) that already exist in a relation. Because Update effectively combines aspects of both removing an old value and inserting a new one, the

constraints it can potentially violate depend heavily on **which specific attribute** is being modified — an update to an ordinary attribute is generally harmless, while an update to a **primary key** or a **foreign key** can trigger the same categories of violations discussed for Insert and Delete.

Specifying an Update

To perform an Update, it is necessary to specify a **condition on the attributes** of the relation, in order to select the particular tuple (or tuples) whose values are to be modified.

Textbook Examples of Update Operations

Example 1 — Acceptable Update (Non-Key Attribute):

Update the **salary** of the EMPLOYEE tuple with Ssn = '999887777' to **28000**.

Result: This update is **acceptable**. Salary is neither a primary key nor a foreign key attribute, so the DBMS only **needs to check** that the new value is of the correct data type and belongs to the correct domain.

Example 2 — Acceptable Update (Valid Foreign Key Change):

Update the **Dno** of the EMPLOYEE tuple with Ssn = '999887777' to **1**.

Result: This update is **acceptable**, since department number 1 is a valid, existing department in the DEPARTMENT relation, so **referential integrity** is preserved.

Example 3 — Unacceptable Update (Referential Integrity Violation):

Update the **Dno** of the EMPLOYEE tuple with Ssn = '999887777' to **7**.

Result: This update is **unacceptable**, because it **violates referential integrity** — there is no department with Dnumber = 7 in the DEPARTMENT relation, so this foreign key value would not correspond to any existing tuple.

Example 4 — Unacceptable Update (Key and Referential Integrity Violation):

Update the **Ssn** of the EMPLOYEE tuple with Ssn = '999887777' to **'987654321'**.

Result: This update is **unacceptable** for **two separate reasons**:

1. It **violates the primary key constraint**, because the new Ssn value '987654321' **already exists as a primary key** in another tuple of EMPLOYEE — this would create a duplicate key.

2. It **violates referential integrity constraints**, because there are other relations that **refer to the existing (old) value** of Ssn for this employee — changing the Ssn would leave those references pointing to a value that no longer exists in this form.

Updating a Non-Key, Non-Foreign-Key Attribute

Updating an attribute that is **neither part of a primary key nor part of a foreign key** usually causes **no problems** at all. In such cases, the DBMS need only perform a simple check to confirm that the **new value is of the correct data type** and belongs to the **correct domain** — as illustrated in Example 1 (updating Salary), where no key or referential concerns arise.

Updating a Primary Key

Modifying a primary key value is conceptually similar to deleting one tuple and inserting another tuple in its place, precisely because the primary key is what is used to **identify** tuples in the first place. Changing it effectively means the "identity" of the tuple, as far as the database is concerned, is changing. As a result, **all the issues discussed earlier for both Insert (Section 5.3.1) and Delete (Section 5.3.2)** come into play simultaneously when a primary key is updated — the new key value must not already exist elsewhere (a key constraint check, as in Insert), and any existing references to the old key value must be handled appropriately (a referential integrity check, as in Delete).

Updating a Foreign Key

If a **foreign key attribute is modified**, the DBMS must ensure that the **new value refers to an existing tuple** in the referenced relation — or, alternatively, that it is **set to NULL**. The same **options** available to handle referential integrity violations caused by Delete — namely **restrict, cascade, and set null/set default** — are similarly available to deal with referential integrity violations caused by Update.

Separate Options for Delete vs. Update

An important point is that, when a referential integrity constraint is specified in the DDL, the DBMS typically allows the user (database designer) to choose **separate, independent options** to handle a violation caused by **Delete** versus a violation caused by **Update** — meaning a single foreign key constraint could, for example, be configured to **cascade on Delete** but **restrict on Update**, or any other combination, depending on what best suits the application's logic.

Important Notes

- Update can potentially trigger the **same categories of violations** as Insert (for primary key changes) and Delete (for foreign key changes).
- Updating a **non-key, non-foreign-key attribute** (e.g., Salary) → only needs a **domain/data-type check**.
- Updating a **primary key** → equivalent to a **delete + insert**; both key constraint and referential integrity issues must be checked.
- Updating a **foreign key** → must ensure the new value **matches an existing PK in the referenced relation, or is NULL**; the same **restrict/cascade/set null** options as Delete apply.
- The DBMS can allow **separate handling options** to be configured for Delete violations versus Update violations, even for the **same** referential integrity constraint.

The Update operation demonstrates how the impact of a constraint violation depends heavily on **which attribute** is being modified. While updating an ordinary attribute like Salary poses no real risk beyond basic domain checking, updating a **primary key** effectively mirrors the combined risks of Delete (breaking existing references) and Insert (creating a potential duplicate), and updating a **foreign key** carries the same referential integrity risks and handling options **restrict, cascade, and set null/set default** as the Delete operation. This close relationship between Update and the other two basic operations underscores why database designers must think carefully about key and foreign key updates, and why DBMSs typically allow separate constraint-handling policies to be defined for Delete and Update independently.

3.5.4 The Transaction Concept (5.3.4)

Introduction

Having examined how the individual Insert, Delete, and Update operations interact with database constraints, it is important to understand how these operations are actually **grouped together and executed** in real database applications. In practice, database

operations are rarely performed in complete isolation instead, they are typically bundled together as part of a larger unit of work known as a **transaction**. This concluding part of Section 5.3 introduces the fundamental concept of a transaction, setting the stage for the more detailed treatment of transaction processing, concurrency, and recovery covered later in the textbook.

Definition of a Transaction

A **database application program** running against a relational database typically executes **one or more transactions**. A **transaction** is defined as an **executing program that includes some database operations**, such as **reading** from the database, or **applying insertions, deletions, or updates** to the database.

Requirement at the End of a Transaction

At the **end of a transaction**, it must leave the database in a **valid, or consistent, state** meaning a state that **satisfies all the constraints** specified on the database schema. This is a critical requirement: regardless of what operations occur *during* a transaction, the database must be left in a **constraint-satisfying state** once the transaction has completed.

Composition of a Transaction

A **single transaction** may involve:

- **Any number of retrieval operations** — these are the operations discussed as part of relational algebra and relational calculus, and later as part of the SQL language.
- **Any number of update operations** — that is, any combination of Insert, Delete, and Update operations, as discussed earlier in this section.

These retrievals and updates, taken **together**, form what is called an **atomic unit of work** against the database. The term *atomic* here signifies that the transaction is treated as a **single, indivisible unit** — either all of its operations are considered to take effect together, or, conceptually, none of them are.

Textbook Example — Bank Withdrawal Transaction

A classic illustrative example of a transaction is a **bank withdrawal**. Such a transaction would typically:

1. **Read** the user's account record from the database.
2. **Check** whether there is a **sufficient balance** to cover the requested withdrawal.

3. **Update** the account record by **subtracting the withdrawal amount** from the balance.

Explanation: This example demonstrates how a transaction naturally combines both a **retrieval operation** (reading the account balance) and an **update operation** (modifying the balance after the withdrawal), all treated together as a single logical unit of work — the withdrawal either completes correctly and consistently, or it should not be considered to have taken effect at all.

Online Transaction Processing (OLTP)

A **large number of commercial applications** running against relational databases operate as **online transaction processing (OLTP) systems**. These systems are capable of **executing transactions at extremely high rates**, reaching **several hundred transactions per second** in many real-world commercial deployments — reflecting how central and performance-critical the transaction concept is to modern relational database usage.

Important Notes

- A **transaction** is an executing program containing database operations — reads, inserts, deletes, and/or updates.
- A transaction **must** leave the database in a **valid/consistent state** at its conclusion, meaning **all schema constraints must be satisfied**.
- A transaction can combine **any number of retrievals and updates**, forming a single **atomic unit of work**.
- The **bank withdrawal** example illustrates a transaction combining a read (check balance) and an update (deduct amount).
- **OLTP systems** are commercial applications that execute transactions at very high rates — sometimes **hundreds per second**.
- Deeper topics such as **transaction processing, concurrent execution, and recovery from failures** are covered in later chapters of the textbook (Chapters 20–22), and are outside the scope of this introductory section.

The transaction concept ties together everything discussed in this chapter — the domain, key, entity integrity, and referential integrity constraints, as well as the Insert, Delete, and Update operations that must respect them. A transaction represents the real-world

unit in which database operations are actually carried out: a bundle of retrievals and updates, executed together as a single atomic unit of work, which must leave the database in a valid, constraint-satisfying state once it completes. The bank withdrawal example neatly illustrates how a single transaction can combine reading data, checking a condition, and updating a record, while the mention of OLTP systems highlights just how frequently and rapidly such transactions occur in real-world commercial database applications. This concept of atomicity and consistency forms the conceptual bridge to the more advanced treatment of transaction processing, concurrency control, and recovery covered later in the textbook.

3.6 Unary Relational Operations: SELECT and PROJECT

3.6.1 The SELECT Operation

Introduction

The **SELECT operation** is one of the two fundamental **unary relational operations** — meaning it operates on a **single relation** — introduced in relational algebra. It is used to **choose a subset of the tuples** from a relation that satisfy a given **selection condition**. Conceptually, the SELECT operation can be understood in several equivalent ways: as a **filter** that retains only those tuples satisfying a qualifying condition, as an operation that **restricts** the tuples of a relation to only those meeting the condition, or as a **horizontal partition** of the relation into two sets of tuples — those that satisfy the condition (and are therefore selected) and those that do not (and are therefore filtered out).

Definition and Notation

The SELECT operation is formally denoted as:

$\sigma_{\langle \text{selection condition} \rangle} (R)$

Here, the symbol σ (**sigma**) represents the SELECT operator, and the **selection condition** is a **Boolean expression** specified on the attributes of relation R. It is important to note that R is, in general, a **relational algebra expression** whose result is itself a relation — in the simplest case, R is simply the name of a database relation. The relation resulting from a SELECT operation has **exactly the same attributes as R**.

Textbook Examples:

To select the EMPLOYEE tuples belonging to department 4:

σ Dno=4 (EMPLOYEE)

To select the EMPLOYEE tuples whose salary exceeds \$30,000:

 σ Salary>30000 (EMPLOYEE)**Structure of the Selection Condition**

The Boolean expression specified in the selection condition is made up of a number of **clauses**, each of one of the following two forms:

- **<attribute name> <comparison op> <constant value>**
- **<attribute name> <comparison op> <attribute name>**

Here, <attribute name> refers to an attribute of R, <comparison op> is normally one of the operators {=, <, ≤, >, ≥, ≠}, and <constant value> is a constant drawn from the attribute's domain. Individual **clauses** can be connected using the standard **Boolean operators AND, OR, and NOT** to build up a more general, compound selection condition.

Textbook Example — Compound Condition:

To select all employees who either work in department 4 and earn over \$25,000 per year, or work in department 5 and earn over \$30,000:

 σ (Dno=4 AND Salary>25000) OR (Dno=5 AND Salary>30000) (EMPLOYEE)**Ordered vs. Unordered Domains**

All the comparison operators in the set {=, <, ≤, >, ≥, ≠} can be applied to attributes whose domains consist of **ordered values**, such as numeric or date domains. Domains of character strings are also considered ordered, based on the **collating sequence** of the characters. However, if the domain of an attribute is a set of **unordered values**, then only the comparison operators {=, ≠} may be used.

Textbook Example — Unordered Domain: The domain **Color** = {'red', 'blue', 'green', 'white', 'yellow', ...} is unordered, since no natural ordering is specified among the various colors. Some domains additionally allow specialized comparison operators — for example, a domain of character strings may permit the operator **SUBSTRING_OF**.

Evaluating the SELECT Condition

The result of a SELECT operation is determined as follows: the selection condition is applied **independently to each individual tuple t** in R. This is achieved by substituting each occurrence of an attribute A_i in the selection condition with its actual value in the tuple, $t[A_i]$. If the resulting condition evaluates to **TRUE**, the tuple t is selected; all selected tuples together form the result of the SELECT operation.

The Boolean operators are interpreted in their standard logical sense:

- **(cond1 AND cond2)** is TRUE if **both** cond1 and cond2 are TRUE; otherwise FALSE.
- **(cond1 OR cond2)** is TRUE if **either or both** of cond1, cond2 are TRUE; otherwise FALSE.
- **(NOT cond)** is TRUE if **cond is** FALSE; otherwise FALSE.

Key Properties of SELECT

- The SELECT operator is **unary** — it is applied to a **single relation**.
- The selection condition is applied to **each tuple individually**; hence, a selection condition **cannot involve more than one tuple** at a time.
- The **degree** (number of attributes) of the relation resulting from a SELECT operation is **the same as the degree of R**.
- The **number of tuples** in the result is always **less than or equal to** the number of tuples in R — formally, $|\sigma_C(R)| \leq |R|$ for any condition C.
- The **fraction of tuples selected** by a given condition is referred to as the **selectivity** of that condition.

Commutativity and Cascading of SELECT

The SELECT operation is **commutative**, meaning:

$$\sigma_{\langle \text{cond1} \rangle} (\sigma_{\langle \text{cond2} \rangle} (R)) = \sigma_{\langle \text{cond2} \rangle} (\sigma_{\langle \text{cond1} \rangle} (R))$$

As a result, a **sequence of SELECT operations can be applied in any order**, without affecting the final result. Furthermore, a **cascade (sequence) of SELECT operations** can always be combined into a **single SELECT operation** using a **conjunctive (AND) condition**:

$\sigma_{\langle \text{cond1} \rangle} (\sigma_{\langle \text{cond2} \rangle} (\dots (\sigma_{\langle \text{condn} \rangle} (R)) \dots)) = \sigma_{\langle \text{cond1} \rangle \text{ AND } \langle \text{cond2} \rangle \text{ AND } \dots \text{ AND } \langle \text{condn} \rangle} (R)$

SQL Equivalent

In SQL, the SELECT condition is typically specified in the **WHERE clause** of a query. For example, the relational algebra expression:

$\sigma_{\text{Dno}=4 \text{ AND Salary}>25000} (\text{EMPLOYEE})$

corresponds to the following SQL query:

```
SELECT * FROM EMPLOYEE
```

```
WHERE Dno=4 AND Salary>25000;
```

- The relational algebra **SELECT** operation is **different from the SQL SELECT clause** — the algebra's **SELECT** chooses *rows*, and is sometimes alternatively called a **RESTRICT** or **FILTER** operation.
- **SELECT** always produces a relation with the **same attributes**, but **fewer or equal tuples**, compared to the input relation *R*.
- **Compound conditions**, combining multiple clauses with AND/OR/NOT, allow arbitrarily complex filtering logic.
- **Selectivity** describes the proportion of tuples a condition retains — useful in query optimization.
- **SELECT** is **commutative** and **cascadable** into a single conjunctive condition — an important simplification rule used in query optimization.

The **SELECT** operation is a foundational unary operator in relational algebra, responsible for filtering a relation down to only those tuples that satisfy a specified Boolean condition, while preserving the full set of attributes. Its behavior — evaluating conditions tuple-by-tuple, supporting compound Boolean expressions, respecting the ordered or unordered nature of attribute domains, and exhibiting commutativity that allows cascaded selections to be merged into a single conjunctive condition — makes it both a conceptually simple and practically powerful tool for query formulation, directly corresponding to the **WHERE** clause in SQL.

3.6.2 – The PROJECT Operation

Introduction

While the SELECT operation filters a relation by choosing certain **rows** and discarding others, the **PROJECT operation** works in the opposite direction along the table: it chooses certain **columns** (attributes) from a relation and discards the rest. If we are interested only in specific attributes of a relation — rather than all of them — the PROJECT operation allows us to "project" the relation onto just those attributes of interest, effectively producing a narrower view of the original relation.

Visualizing PROJECT

If the SELECT operation can be visualized as a **horizontal partition** of a relation (splitting it into selected and non-selected rows), the result of the PROJECT operation can be visualized as a **vertical partition** of the relation into two parts: one containing the **needed columns** (which forms the result of the operation), and the other containing the **discarded columns**.

Definition and Notation

The general form of the PROJECT operation is:

$\pi \langle \text{attribute list} \rangle (R)$

Here, the symbol π (**pi**) represents the PROJECT operator, and $\langle \text{attribute list} \rangle$ is the desired sublist of attributes drawn from the attributes of relation R. As with SELECT, R is, in general, a relational algebra expression whose result is a relation — in the simplest case, just the name of a database relation.

The result of the PROJECT operation contains **only the attributes specified in $\langle \text{attribute list} \rangle$** , appearing in the **same order** as they are listed. Consequently, the **degree** of the resulting relation is equal to the **number of attributes in $\langle \text{attribute list} \rangle$** .

Textbook Example:

To list each employee's first name, last name, and salary:

$\pi \text{ Lname, Fname, Salary (EMPLOYEE)}$

Duplicate Elimination

If the attribute list used in a PROJECT operation includes **only nonkey attributes** of R, **duplicate tuples are likely to occur** in the raw result, since two or more tuples of R may agree on the values of just those particular attributes. The PROJECT operation, however,

removes any duplicate tuples from its result — this process is known as **duplicate elimination**. Since a formal relation is defined as a *set* of tuples, and sets cannot contain duplicate elements, this ensures that the output of PROJECT is always a **valid relation**, consisting of a set of *distinct* tuples.

Textbook Example:

π Sex, Salary (EMPLOYEE)

Explanation: In this example, the tuple <'F', 25000> appears only **once** in the final result, even though this exact combination of Sex and Salary values may occur in **two or more** separate tuples of the original EMPLOYEE relation. The duplicate copies are eliminated so that the result remains a proper set.

Important Note: Duplicate elimination requires **sorting or some other technique** to detect and remove duplicate tuples, which adds additional **processing overhead** to the PROJECT operation. If duplicates were *not* eliminated, the result would instead be a **multiset (or bag) of tuples** rather than a true set — such multisets are **not permitted** in the *formal* relational model, though they **are allowed in SQL**.

Key Properties of PROJECT

- The **number of tuples** in the relation resulting from a PROJECT operation is always **less than or equal to** the number of tuples in R.
- If the projection list happens to be a **superkey of R** — that is, it includes some key of R — then the resulting relation has **exactly the same number of tuples** as R (since no duplicates can occur when a key is included, every tuple remains distinct).
- The following identity holds:

$$\pi\langle\text{list1}\rangle (\pi\langle\text{list2}\rangle (R)) = \pi\langle\text{list1}\rangle (R)$$

provided that **<list2> contains all the attributes in <list1>**; if this condition is not met, the left-hand side expression is **not valid**.

- **Important Note:** Unlike SELECT, **commutativity does NOT hold for PROJECT**. That is, the order in which nested PROJECT operations are applied matters, and they cannot generally be freely reordered the way SELECT conditions can.

SQL Equivalent

In SQL, the PROJECT attribute list is specified in the **SELECT clause** of a query. For example, the relational algebra expression:

π Sex, Salary (EMPLOYEE)

corresponds to the following SQL query:

```
SELECT DISTINCT Sex, Salary
FROM EMPLOYEE;
```

Important Note: If the keyword **DISTINCT** is removed from this SQL query, then **duplicates will NOT be eliminated** in the SQL result. This "no duplicate elimination" option is **not available** in the formal relational algebra as originally defined — although the algebra can be *extended* to support this behavior by allowing relations to be treated as multisets, a topic not discussed in detail in the basic model.

- PROJECT selects **columns**, not rows — a **vertical partition**, in contrast to SELECT's horizontal partition.
- The **degree** of the PROJECT result equals the **number of attributes** in the attribute list.
- **Duplicate elimination** is a defining feature of PROJECT in the formal relational algebra, ensuring the result remains a valid set-based relation.
- If the attribute list forms (or includes) a **superkey**, no duplicates can arise, and the tuple count is preserved.
- **PROJECT is NOT commutative**, unlike SELECT — this is an important distinguishing property between the two unary operators.
- SQL's **DISTINCT** keyword corresponds directly to the duplicate-elimination behavior of the formal PROJECT operation; **omitting DISTINCT** in SQL breaks from the strict relational algebra definition and permits duplicate rows.

The PROJECT operation complements the SELECT operation by narrowing a relation along its **columns** rather than its rows, retaining only the attributes specified in the projection list while discarding the rest. A defining and often-examined property of PROJECT is **duplicate elimination**, which ensures the result remains a true set of distinct tuples in keeping with the formal definition of a relation — though this comes at the cost of additional processing to detect and remove duplicates. Unlike SELECT, PROJECT is **not commutative**, and its degree is directly determined by the size of the

attribute list supplied. In SQL, this behavior is mirrored by the SELECT clause combined with the DISTINCT keyword, with the important caveat that omitting DISTINCT departs from the strict formal definition and allows duplicate rows to remain.

3.6.3 Sequences of Operations and the RENAME Operation

Introduction

Individual SELECT and PROJECT operations, applied on their own, can only answer relatively simple queries. Most real-world queries, however, require **several relational algebra operations to be applied one after another** — for instance, first filtering a relation's rows with SELECT, and then narrowing its columns with PROJECT. This raises a practical question: how **should** such multi-step queries be written and organized? This section introduces **two approaches** for combining operations — writing a single nested expression, or breaking the query into named intermediate steps — along with the formal **RENAME operation**, which allows relations and their attributes to be given new, meaningful names.

Two Approaches to Sequencing Operations

When several relational algebra operations must be applied in sequence, there are **two general approaches**:

1. **In-line (nested) expression:** Write the operations as a **single relational algebra expression**, by nesting one operation inside another.
2. **Step-by-step with intermediate relations:** Apply **one operation at a time**, creating and naming **intermediate result relations** along the way.

Textbook Example — Two Approaches Compared

Query: Retrieve the first name, last name, and salary of all employees who work in department number 5.

Approach 1 — In-line Expression:

π Fname, Lname, Salary (σ Dno=5 (EMPLOYEE))

Here, the SELECT operation is applied first (innermost), filtering EMPLOYEE down to only department-5 tuples, and the PROJECT operation is then applied to the result, retaining only the Fname, Lname, and Salary attributes.

Approach 2 — Step-by-Step with Named Intermediate Relations:

DEP5_EMPS $\leftarrow \sigma \text{Dno}=5$ (EMPLOYEE)

RESULT $\leftarrow \pi \text{Fname, Lname, Salary}$ (DEP5_EMPS)

Explanation: Here, the **assignment operation**, denoted by the symbol $\leftarrow$ (**left arrow**), is used to explicitly name each intermediate result. The first step creates the named relation DEP5_EMPS, holding only department-5 employees; the second step then projects this intermediate relation down to just the three required attributes, storing the final answer in RESULT.

Important Note: It is often **simpler to break down a complex sequence of operations** by introducing named intermediate relations, rather than attempting to write one large, deeply nested single expression — particularly as queries grow more complex, involving operations such as UNION and JOIN.

Renaming Attributes Using Intermediate Relations

The technique of naming intermediate relations can also be used to **rename the attributes** of those relations. This is done simply by listing the new attribute names in parentheses immediately after the new relation name.

Textbook Example:

TEMP $\leftarrow \sigma \text{Dno}=5$ (EMPLOYEE)

R(First_name, Last_name, Salary) $\leftarrow \pi \text{Fname, Lname, Salary}$ (TEMP)

Explanation: Here, TEMP holds the intermediate result of filtering EMPLOYEE by department 5. The second step then projects the required attributes from TEMP, and simultaneously **renames** them — the resulting relation is called R, with its three attributes explicitly renamed to First_name, Last_name, and Salary respectively, instead of retaining the original attribute names Fname, Lname, Salary.

Default Attribute Naming (No Renaming)

If **no renaming** is applied:

- For a **SELECT** operation, the attributes of the resulting relation are the **same as those in the original relation**, and appear in the **same order**.
- For a **PROJECT** operation, the resulting relation has the **same attribute names as those listed in the projection list**, appearing in the **same order in which they were listed**.

The Formal RENAME Operation

In addition to the informal renaming technique shown above (naming attributes alongside an intermediate relation), relational algebra also defines a **formal RENAME operation**, which can rename the **relation name**, the **attribute names**, or **both**, and is itself treated as a **unary operator**.

When applied to a relation R of degree n , the general RENAME operation is denoted using any of the **following three forms**:

- $\rho S(B_1, B_2, \dots, B_n)(R)$ — renames both the **relation** and its **attributes**.
- $\rho S(R)$ — renames **only the relation name**.
- $\rho(B_1, B_2, \dots, B_n)(R)$ — renames **only the attributes**.

Here, the symbol ρ (**rho**) denotes the RENAME operator, S is the new relation name, and $B_1, B_2, \dots, B_n$ are the new attribute names. If the original attributes of R are $(A_1, A_2, \dots, A_n)$, listed in that specific order, then under renaming, each A_i is **renamed to the corresponding B_i** .

SQL Equivalent

In SQL, a single query typically represents an entire, complex relational algebra expression combining multiple operations. **Renaming in SQL** is accomplished through **aliasing**, using the keyword **AS**.

Textbook Example:

```
SELECT E.Fname AS First_name, E.Lname AS Last_name, E.Salary AS Salary
FROM EMPLOYEE AS E
WHERE E.Dno=5;
```

Explanation: Here, the EMPLOYEE relation is aliased as **E** (renaming the relation), while the Fname, Lname, and Salary attributes are aliased as First_name, Last_name, and Salary respectively (renaming the attributes) — directly mirroring the formal ρ (RENAME) operation of relational algebra, all expressed within a single SQL query.

Important Notes

- Two approaches exist for sequencing multiple operations: **in-line (nested) expressions** and **step-by-step with named intermediate relations**.
- The **assignment operation** ($\leftarrow$) is used to name intermediate results when using the step-by-step approach.
- Renaming attributes can be done **informally**, by listing new names in parentheses alongside an intermediate relation's name, or **formally**, using the ρ (RENAME) operator.
- The RENAME operator ρ has **three forms**: renaming both relation and attributes, renaming only the relation, or renaming only the attributes.
- **Without renaming**, SELECT preserves the original attribute names and order, while PROJECT uses the attribute names and order exactly as given in its attribute list.
- SQL achieves renaming through **aliasing with the AS keyword**, directly corresponding to the formal RENAME operation.

While SELECT and PROJECT are individually simple operations, real-world queries typically require them to be combined in sequence — either through compact, nested in-line expressions, or through a clearer, step-by-step sequence of named intermediate relations connected using the assignment operation. The formal RENAME operation, denoted by ρ , extends this naming capability into a proper relational algebra operator, allowing both relation names and attribute names to be changed as needed — a capability that becomes especially valuable in more complex operations such as UNION and JOIN, where matching or distinguishing attribute names across relations is essential. In SQL, this entire naming and sequencing capability is captured elegantly through table and column aliasing using the AS keyword.

3.7 Relational Algebra Operations from Set Theory

3.7.1 The UNION, INTERSECTION, and MINUS Operations

Introduction

The next group of relational algebra operations is drawn directly from **standard mathematical set theory**. Since a relation is formally defined as a *set* of tuples, it becomes natural to apply the familiar set-theoretic operations **UNION**, **INTERSECTION**, and **SET DIFFERENCE (MINUS)** to combine or compare the tuples of two relations. These operations allow queries that require merging results from multiple sub-queries, finding common tuples between two relations, or identifying tuples present in one relation but absent from another.

Textbook Motivating Example

Query: Retrieve the Social Security numbers of all employees who **either work in department 5, or directly supervise an employee who works in department 5**.

Relational Algebra:

$\text{DEP5_EMPS} \leftarrow \sigma_{\text{Dno}=5}(\text{EMPLOYEE})$ $\text{RESULT1} \leftarrow \pi_{\text{Ssn}}(\text{DEP5_EMPS})$
 $\text{RESULT2}(\text{Ssn}) \leftarrow \pi_{\text{Super_ssn}}(\text{DEP5_EMPS})$ $\text{RESULT} \leftarrow \text{RESULT1} \cup \text{RESULT2}$

Explanation: Here, **RESULT1** holds the Ssn of all employees who work in department 5, while **RESULT2** holds the Ssn of all employees who **directly supervise** someone working in department 5 (obtained via the Super_ssn attribute, then renamed to Ssn for compatibility). The **UNION** operation then combines these two sets of Ssn values, producing all tuples present in **either RESULT1 or RESULT2 or both**, while **eliminating any duplicates** — so a supervisor who also happens to work in department 5 (such as Ssn '333445555') appears **only once** in the final RESULT.

As a **single relational algebra expression**, this entire query can be written as:

$\text{RESULT} \leftarrow \pi_{\text{Ssn}}(\sigma_{\text{Dno}=5}(\text{EMPLOYEE})) \cup \pi_{\text{Super_ssn}}(\sigma_{\text{Dno}=5}(\text{EMPLOYEE}))$

Set-Theoretic Operations — Overview

Several set-theoretic operations are used to merge the elements of two sets in different ways, including **UNION**, **INTERSECTION**, and **SET DIFFERENCE** (also called **MINUS** or **EXCEPT**). These are all **binary operations** — that is, each is applied to exactly **two sets (of tuples)**.

Union Compatibility (Type Compatibility)

When these set operations are adapted for use with relational databases, an important restriction applies: the **two relations** on which any of these three operations is applied **must have the same type of tuples** — a requirement known as **union compatibility** (or **type compatibility**).

Formal Definition: Two relations $R(A_1, A_2, \dots, A_n)$ and $S(B_1, B_2, \dots, B_n)$ are said to be **union compatible** (or **type compatible**) if:

1. They have the **same degree n** (the same number of attributes), **and**
2. **$\text{dom}(A_i) = \text{dom}(B_i)$ for $1 \leq i \leq n$** — meaning each corresponding pair of attributes shares the **same domain**.

In other words, the two relations **must** have the same number of attributes, with each corresponding attribute pair **drawn from** an identical domain.

Definitions of UNION, INTERSECTION, and SET DIFFERENCE

Given two union-compatible relations R and S :

- **UNION ($R \cup S$):** The result is a relation containing **all tuples that are either in R or in S , or in both R and S . Duplicate tuples are eliminated.**
- **INTERSECTION ($R \cap S$):** The result is a relation containing **all tuples that are in both R and S .**
- **SET DIFFERENCE / MINUS ($R - S$):** The result is a relation containing **all tuples that are in R but not in S .**

Convention on Attribute Names: By convention, the resulting relation from any of these three operations retains the **same attribute names as the first relation, R** . If different attribute names are desired for the result, the **RENAME operator** can always be applied afterward.

Textbook Example — STUDENT and INSTRUCTOR

Consider two union-compatible relations, **STUDENT** and **INSTRUCTOR**, whose tuples represent the names of students and instructors, respectively:

- **$\text{STUDENT} \cup \text{INSTRUCTOR}$** — produces the names of **all** students and instructors combined, with duplicate names (people who are both) appearing **only once**.

- **STUDENT $\cap$ INSTRUCTOR** — produces only the names of people who are **both** students **and** instructors.
- **STUDENT – INSTRUCTOR** — produces the names of students who are **not** instructors.
- **INSTRUCTOR – STUDENT** — produces the names of instructors who are **not** students.

Properties of UNION and INTERSECTION

Both **UNION** and **INTERSECTION** are **commutative** operations:

$$R \cup S = S \cup R \text{ and } R \cap S = S \cap R$$

Both operations are also **associative**, which means they can be treated as **n-ary operations**, applicable across any number of relations (not just two):

$$R \cup (S \cup T) = (R \cup S) \cup T \text{ and } (R \cap S) \cap T = R \cap (S \cap T)$$

Property of MINUS (SET DIFFERENCE)

Unlike **UNION** and **INTERSECTION**, the **MINUS operation** is **NOT commutative**. In general:

$$R - S \neq S - R$$

This is intuitively clear from the **STUDENT/INSTRUCTOR** example: the set of students who are not instructors (**STUDENT – INSTRUCTOR**) is generally a completely different set from the set of instructors who are not students (**INSTRUCTOR – STUDENT**).

Expressing INTERSECTION Using UNION and MINUS

It is worth noting that **INTERSECTION** can be expressed in terms of **UNION** and **SET DIFFERENCE**, as follows:

$$R \cap S = ((R \cup S) - (R - S)) - (S - R)$$

This identity demonstrates that **INTERSECTION** is not strictly a "primitive" operation — it can be derived entirely from **UNION** and **MINUS**, though it is retained as a separate, named operation for clarity and convenience.

SQL Equivalent

In SQL, **three operations** — **UNION**, **INTERSECT**, and **EXCEPT** — correspond directly to the three set operations described here. In addition, SQL also supports **multiset variants** — **UNION ALL**, **INTERSECT ALL**, and **EXCEPT ALL** — which **do not eliminate duplicates**, unlike their standard counterparts.

Important Notes

- All three operations — **UNION**, **INTERSECTION**, **SET DIFFERENCE** — are **binary** and require the two input relations to be **union compatible** (same degree, same corresponding domains).
- **UNION** eliminates duplicates; **INTERSECTION** returns only common tuples; **MINUS** returns tuples in R but absent from S.
- The result's attribute names follow the convention of using **R's attribute names** (the first relation).
- **UNION** and **INTERSECTION** are **commutative AND associative** — making them usable as effective n-ary operations.
- **MINUS** is **NOT commutative** — $R - S \neq S - R$, in general.
- **INTERSECTION** can be derived from **UNION** and **MINUS** using the identity $R \cap S = ((R \cup S) - (R - S)) - (S - R)$.
- SQL equivalents: **UNION**, **INTERSECT**, **EXCEPT** (duplicate-eliminating) and **UNION ALL**, **INTERSECT ALL**, **EXCEPT ALL** (duplicate-preserving).

The **UNION**, **INTERSECTION**, and **SET DIFFERENCE (MINUS)** operations extend classical set theory directly into relational algebra, enabling queries that combine, compare, or contrast tuples across two relations. Their applicability, however, is strictly governed by the requirement of **union compatibility**, ensuring that the relations being combined share an identical structure of attributes and domains. While **UNION** and **INTERSECTION** share the convenient mathematical properties of commutativity and associativity — allowing them to be extended naturally to more than two relations — **MINUS** lacks commutativity, making the order of operands significant. These operations, mirrored closely by SQL's **UNION**, **INTERSECT**, and **EXCEPT** (along with their **ALL** variants), form an essential toolkit for combining data drawn from structurally compatible relations.

3.7.2 CARTESIAN PRODUCT (CROSS PRODUCT) Operation

Introduction

The final set-theoretic operation discussed in this section is the **CARTESIAN PRODUCT**, also known as the **CROSS PRODUCT** or **CROSS JOIN**, denoted by the symbol $\times$. Like UNION, INTERSECTION, and MINUS, this is also a **binary set operation** — but unlike those three operations, the relations on which the CARTESIAN PRODUCT is applied **do not need to be union compatible**. Instead of merging or comparing tuples of similar structure, the CARTESIAN PRODUCT **combines** tuples from two relations in every **possible** way, producing a much larger relation whose attributes come from **both** original relations together.

Definition and Working

In its binary form, the CARTESIAN PRODUCT operation produces a new tuple by **combining every member (tuple) from one relation with every member (tuple) from the other relation**. Formally, given two relations $R(A_1, A_2, \dots, A_n)$ and $S(B_1, B_2, \dots, B_m)$, the result of $R \times S$ is a new relation Q of degree $n + m$, with attributes:

$Q(A_1, A_2, \dots, A_n, B_1, B_2, \dots, B_m)$

listed in that specific order. The resulting relation Q contains **one tuple for each possible combination** of tuples — one tuple taken from R and one tuple taken from S . Consequently, if R has n_R tuples (written $|R| = n_R$) and S has n_S tuples, then the resulting relation $R \times S$ will have $n_R \times n_S$ tuples — the product of the two tuple counts.

The n-ary CARTESIAN PRODUCT

The concept of the binary CARTESIAN PRODUCT can be extended to an **n-ary CARTESIAN PRODUCT** operation, which produces new tuples by **concatenating all possible combinations of tuples drawn from n underlying relations** simultaneously, rather than just two.

Meaningfulness of CARTESIAN PRODUCT

An important observation about the CARTESIAN PRODUCT is that, **applied by itself, it is generally meaningless** — combining every tuple of one relation with every tuple of another, without any regard to whether the combination makes logical sense, typically

produces a very large relation full of nonsensical tuple combinations. The **CARTESIAN PRODUCT** becomes **useful only when followed by a SELECT operation** that matches values of attributes coming from the two component relations, thereby filtering the massive combined result down to only the **meaningful** combinations.

Textbook Example — Female Employees and Their Dependents

Query: Retrieve a list of names of each female employee's dependents.

Relational Algebra:

$$\begin{aligned} \text{FEMALE_EMPS} &\leftarrow \sigma_{\text{Sex}='F'}(\text{EMPLOYEE}) \\ \text{EMP_DEPENDENTS} &\leftarrow \pi_{\text{Fname, Lname, Ssn}}(\text{FEMALE_EMPS}) \\ \text{EMP_DEPENDENTS} &\leftarrow \text{EMP_DEPENDENTS} \times \text{DEPENDENT} \\ \text{ACTUAL_DEPENDENTS} &\leftarrow \sigma_{\text{Ssn}=\text{Essn}}(\text{EMP_DEPENDENTS}) \\ \text{RESULT} &\leftarrow \pi_{\text{Fname, Lname, Dependent_name}}(\text{ACTUAL_DEPENDENTS}) \end{aligned}$$

Step-by-Step Explanation:

1. **FEMALE_EMPS** — selects only the **EMPLOYEE** tuples where **Sex = 'F'**.
2. **EMP_NAMES** — projects just the **Fname, Lname, and Ssn** attributes of these female employees.
3. **EMP_DEPENDENTS** — applies the **CARTESIAN PRODUCT** between **EMP_NAMES** and the entire **DEPENDENT** relation. This step combines **every** female employee tuple with **every** **DEPENDENT** tuple, regardless of whether that dependent actually belongs to that employee — producing a very large, mostly meaningless relation, since **every dependent gets combined with every female employee**.
4. **ACTUAL_DEPENDENTS** — applies a **SELECT** operation with the condition **Ssn = Essn**, filtering **EMP_DEPENDENTS** down to **only those combined tuples where the employee's Ssn actually matches the dependent's Essn** — that is, only the combinations representing a **real** employee-dependent relationship.
5. **RESULT** — finally projects just the **Fname, Lname, and Dependent_name** attributes from **ACTUAL_DEPENDENTS**, giving the clean, meaningful final answer.

Key Observation — CARTESIAN PRODUCT Can Produce Meaningless Results

The **EMP_DEPENDENTS** relation in this example is explicitly highlighted in the textbook as a **good illustration of a case where relational algebra can be correctly applied, yet still yield results that make no sense at all** — since it pairs every female employee with every dependent in the database, most of these pairings do not correspond to any real-world relationship. This underscores an important responsibility: **it is up to the user to ensure that only meaningful operations are applied to relations**, since the algebra itself will not prevent nonsensical combinations from being formed.

Relationship to JOIN

The **CARTESIAN PRODUCT** creates tuples containing the **combined attributes of two relations**. As shown in the example above, we can retrieve only the **related** tuples from the two relations by specifying an appropriate **selection condition after the CARTESIAN PRODUCT** — **exactly** as done with the $Ssn = Essn$ condition. Because this specific sequence — **CARTESIAN PRODUCT followed by SELECT** — is used so commonly to combine related tuples from two relations, a **special operation called JOIN** was introduced specifically to **express this entire sequence as a single operation**. The JOIN operation is discussed in the section that follows.

SQL Equivalent

In SQL, the **CARTESIAN PRODUCT** can be realized using the **CROSS JOIN** option within joined tables. Alternatively, if a query's **FROM clause lists two tables**, and there is **no corresponding join condition specified in the WHERE clause**, the result of the query will **also be the CARTESIAN PRODUCT** of the two tables.

Important Notes

- **CARTESIAN PRODUCT ($\times$)** is a **binary operation**, but unlike **UNION/INTERSECTION/MINUS**, it does **NOT require union compatibility**.
- The result's **degree** = $n + m$ (sum of the degrees of both input relations); the result's **tuple count** = $nR \times nS$ (product of both tuple counts).
- **CARTESIAN PRODUCT alone is generally meaningless**; it becomes useful only when followed by a **SELECT** that matches related attributes across the two relations.
- The sequence **CARTESIAN PRODUCT + SELECT** is common enough that it was formalized into a single operation called **JOIN**.

- SQL equivalent: **CROSS JOIN**, or simply listing two tables in FROM **without** a join condition in WHERE.

The CARTESIAN PRODUCT operation combines every tuple of one relation with every tuple of another, producing a relation whose degree is the sum of the two input degrees and whose tuple count is the product of the two input tuple counts. While mathematically well-defined and always computable regardless of union compatibility, the CARTESIAN PRODUCT is rarely useful on its own, since it indiscriminately pairs unrelated tuples together — as vividly illustrated by the EMP_DEPENDENTS example, where every female employee is paired with every dependent in the database, most of these pairings being meaningless. Its true value emerges only when immediately followed by a SELECT operation that filters the result down to genuinely related tuple combinations — a pattern so common that it motivated the creation of the JOIN operation, discussed next, which expresses this entire CARTESIAN PRODUCT-plus-SELECT sequence as a single, unified relational algebra operation.

3.8 Binary Relational Operations: JOIN and DIVISION

3.8.1 The JOIN Operation

Introduction

The **JOIN operation**, denoted by the symbol $\bowtie$, is used to **combine related tuples from two relations into single, "longer" tuples**. This operation is of central importance to any relational database containing more than a single relation, because it is precisely the JOIN operation that allows us to **process relationships among relations** — reconstructing meaningful, connected information that had been deliberately split across multiple tables during database design.

Motivating Example

Query: Retrieve the name of the manager of each department.

To obtain the manager's name, it is necessary to **combine each department tuple with the employee tuple** whose Ssn value matches the Mgr_ssn value stored in that department tuple. This is accomplished using the JOIN operation, followed by a projection over the necessary attributes:

DEPT_MGR $\leftarrow$ **DEPARTMENT** $\bowtie$ **Mgr_ssn=Ssn EMPLOYEE**

RESULT $\leftarrow \pi$ **Dname, Lname, Fname (DEPT_MGR)**

Explanation: Here, **Mgr_ssn** is a **foreign key** of the **DEPARTMENT** relation, referencing **Ssn**, the primary key of the **EMPLOYEE** relation. This underlying **referential integrity constraint** is precisely what guarantees that matching tuples will actually be found in the referenced **EMPLOYEE** relation during the **JOIN**.

JOIN as CARTESIAN PRODUCT + SELECT

The **JOIN** operation can, in principle, be specified as a **CARTESIAN PRODUCT operation followed by a SELECT operation**. However, **JOIN** is considered important enough to be treated as its own distinct operation, precisely because it is used **so frequently** when specifying database queries.

Textbook Example — Replacing CARTESIAN PRODUCT + SELECT with JOIN:

Recall the earlier **CARTESIAN PRODUCT** example:

EMP_DEPENDENTS $\leftarrow$ **EMPNAMES** $\times$ **DEPENDENT**
ACTUAL_DEPENDENTS $\leftarrow \sigma$ **Ssn=Essn (EMP_DEPENDENTS)**

These two operations can be **replaced with a single JOIN operation**, as follows:

ACTUAL_DEPENDENTS $\leftarrow$ **EMPNAMES** $\bowtie$ **Ssn=Essn DEPENDENT**

This demonstrates directly how **JOIN** **condenses** the common "combine, then filter" pattern into a single, cleaner operation.

General Form of JOIN

The general form of a **JOIN** operation on two relations **R(A1, A2, ..., An)** and **S(B1, B2, ..., Bm)** is:

R $\bowtie$ **<join condition> S**

The result of the **JOIN** is a relation **Q** with **n + m attributes**, **Q(A1, A2, ..., An, B1, B2, ..., Bm)**, in that order. **Q** contains **one tuple for each combination of tuples** — one from **R** and one from **S** — **whenever that combination satisfies the join condition**.

Key Difference: JOIN vs. CARTESIAN PRODUCT

This is the **main difference** between **CARTESIAN PRODUCT** and **JOIN**:

- In **JOIN**, only combinations of tuples **satisfying the join condition** appear in the result.
- In **CARTESIAN PRODUCT**, **all** combinations of tuples are included in the result, regardless of whether they make sense.

The join condition is specified on attributes drawn from the two relations R and S, and it is **evaluated for each combination of tuples**. Each tuple combination for which the join condition evaluates to **TRUE** is included in the resulting relation Q as a single, combined tuple.

THETA JOIN

A **general join condition** takes the form:

<condition> AND <condition> AND ... AND <condition>

where each individual <condition> has the form **$A_i \theta B_j$** , with A_i being an attribute of R, B_j an attribute of S, A_i and B_j sharing the **same domain**, and θ (**theta**) being one of the comparison operators $\{=, <, \leq, >, \geq, \neq\}$. A JOIN operation using such a general join condition is called a **THETA JOIN**.

Important Note: Tuples whose join attributes are **NULL**, or for which the join condition evaluates to **FALSE**, **do not appear in the result**. In this sense, the JOIN operation **does not necessarily preserve all the information** present in the participating relations — tuples that fail to combine with any matching tuple in the other relation are simply **excluded** from the output.

Important Notes

- JOIN is denoted by $\bowtie$, and combines tuples from two relations based on a specified **join condition**.
- The result's **degree** = **n + m** (sum of both relations' degrees) — same as CARTESIAN PRODUCT.
- Unlike CARTESIAN PRODUCT, JOIN's result **only includes combinations that satisfy the join condition**.
- A **THETA JOIN** allows any of the comparison operators $\{=, <, \leq, >, \geq, \neq\}$ in its join condition, not just equality.

- JOIN can always be expressed as **CARTESIAN PRODUCT** followed by **SELECT**, but is retained as a separate operation for **convenience and frequency of use**.
- JOIN **does not preserve unmatched tuples** — this is an important limitation later addressed by **OUTER JOIN** operations (Section 8.4.4).

The JOIN operation stands as one of the most essential operations in relational algebra, enabling related tuples from two separate relations to be combined into single, meaningful tuples based on a specified join condition. While logically equivalent to a CARTESIAN PRODUCT followed by a SELECT operation, JOIN is retained as a distinct, named operation precisely because of how frequently it is applied in real database queries — from combining departments with their managers, to combining employees with their dependents. Its general form, the THETA JOIN, supports any comparison operator in its join condition, though the vast majority of practical joins rely on equality comparisons, as discussed in the next part covering EQUIJOIN and NATURAL JOIN.

3.9 Binary Relational Operations: JOIN and DIVISION

Introduction

While the general THETA JOIN allows any comparison operator in its join condition, the **overwhelming majority of practical joins** rely exclusively on **equality comparisons**. This special, extremely common case of JOIN is given its own name — the **EQUIJOIN** — and, building further on this, a refined variant called **NATURAL JOIN** was introduced to eliminate a specific inefficiency that EQUIJOIN always produces. This part also examines the broader theoretical question of which relational algebra operations are truly *necessary*, versus which are included purely for **convenience**.

EQUIJOIN

The **most common use of JOIN** involves join conditions containing **only equality comparisons**. Such a JOIN — where the **only comparison operator used is =** — is called an **EQUIJOIN**. Both of the JOIN examples discussed in Part 1 (DEPT_MGR and ACTUAL_DEPENDENTS) were, in fact, EQUIJOINS.

Key Observation: In the result of an EQUIJOIN, there will always be **one or more pairs of attributes that have identical values in every tuple**. For example, in the

DEPT_MGR relation shown earlier, the values of the attributes **Mgr_ssn** and **Ssn** are **identical in every tuple** — this happens precisely *because* the equality join condition specified on these two attributes forces their values to match in every resulting tuple.

NATURAL JOIN

Since, in every EQUIJOIN result, **one of each pair of attributes with identical values is superfluous** (redundant — carrying no new information beyond what the other attribute in the pair already shows), a new operation called **NATURAL JOIN** — denoted by the symbol $*$ — was created specifically to **eliminate this superfluous, duplicate attribute** from the EQUIJOIN result.

Important Note: NATURAL JOIN is essentially an EQUIJOIN followed by the **removal of the superfluous (duplicate) attribute**.

The **standard definition of NATURAL JOIN** requires that the **two join attributes** (or each corresponding pair of join attributes) **have the same name in both relations**. If this naming condition is **not** already satisfied, a **RENAME operation** must first be applied to make the attribute names match.

Textbook Example — PROJECT and DEPARTMENT

Query: Combine each PROJECT tuple with the DEPARTMENT tuple that controls the project.

Since the PROJECT relation's foreign key is named **Dnum**, while the corresponding primary key in DEPARTMENT is named **Dnumber**, these names **do not match** — so a RENAME must be applied first:

PROJ_DEPT $\leftarrow$ **PROJECT** $*$ ρ (Dname, Dnum, Mgr_ssn, Mgr_start_date) (DEPARTMENT)

Explanation: Here, DEPARTMENT's Dnumber attribute is renamed to **Dnum** (matching PROJECT's foreign key name), and then the NATURAL JOIN ($*$) is applied. The same query can equivalently be written in **two separate steps**, using an intermediate relation:

DEPT $\leftarrow$ ρ (Dname, Dnum, Mgr_ssn, Mgr_start_date) (DEPARTMENT)
PROJ_DEPT $\leftarrow$ **PROJECT** $*$ **DEPT**

Explanation: The attribute **Dnum** is called the **join attribute** for this NATURAL JOIN operation, because it is the **only attribute sharing the same name** in both relations after

renaming. In the resulting **PROJ_DEPT** relation, each tuple combines a PROJECT tuple with the DEPARTMENT tuple for the department that controls it — but crucially, **only one copy** of the join attribute value is retained in the result, eliminating the redundancy that an EQUIJOIN would have left behind.

Example Without Renaming — DEPARTMENT and DEPT_LOCATIONS

If the attributes on which the NATURAL JOIN is specified **already share the same name** in both relations, **renaming is unnecessary**. For example, to apply a NATURAL JOIN on the **Dnumber** attribute of DEPARTMENT and DEPT_LOCATIONS (which already share this exact attribute name), it is sufficient to simply write:

DEPT_LOCS $\leftarrow$ DEPARTMENT * DEPT_LOCATIONS

Explanation: The resulting relation combines each department with its associated locations, producing **one tuple for each location** a department has.

General Rule for Constructing the NATURAL JOIN Condition

In general, the join condition for a NATURAL JOIN is constructed by **equating each pair of join attributes that share the same name** in the two relations, and then **combining these individual equality conditions using AND**. There can be a **list of join attributes** from each relation, and **each corresponding pair must share the same name** for this automatic matching to work correctly.

Result Size of JOIN

If **no combination of tuples satisfies the join condition**, the result of a JOIN is simply an **empty relation with zero tuples**. In general, if R has **nR tuples** and S has **nS tuples**, the result of a JOIN operation $R \bowtie \langle \text{join condition} \rangle S$ will have **between zero and $nR \times nS$ tuples**. The **expected size of the join result, divided by the maximum possible size ($nR \times nS$)**, gives a ratio known as the **join selectivity**, which is considered a property of each specific join condition.

Important Note: If **no join condition** is specified at all, then **all combinations of tuples qualify**, and the JOIN operation **degenerates into a CARTESIAN PRODUCT** (also called CROSS PRODUCT or CROSS JOIN).

Inner Joins vs. Outer Joins (Preview)

A single JOIN operation is used to combine data from two relations so that related information can be presented together in a single table. These standard JOIN operations

are also referred to as **inner joins**, specifically to distinguish them from a different variation called **outer joins** (covered later, in Section 8.4.4). Informally, an **inner join** is a type of **match-and-combine** operation, formally defined as a **combination of CARTESIAN PRODUCT and SELECTION**.

Important Note: A join can sometimes be specified **between a relation and itself** — this special case is discussed further in Section 8.4.3 (recursive relationships).

n-Way Joins

The NATURAL JOIN or EQUIJOIN operation can also be applied across **multiple tables simultaneously**, leading to what is called an **n-way join**.

Textbook Example — Three-Way Join:

((PROJECT ⋈ Dnum=Dnumber DEPARTMENT) ⋈ Mgr_ssn=Ssn EMPLOYEE)

Explanation: This expression first combines each project tuple with its **controlling department** tuple, and then combines that intermediate result with the **employee tuple representing the department's manager**. The final, consolidated relation contains combined **project–department–manager** information in a single tuple per project.

JOIN in SQL

In SQL, JOIN can be realized in **several different ways**:

1. **Specifying join conditions in the WHERE clause**, alongside any other selection conditions — this is the **most common** approach.
2. **Using nested relations/subqueries** — a somewhat less direct approach.
3. **Using joined tables** (explicit JOIN syntax) — this construct was **added to SQL2** specifically to let users explicitly specify all the various types of joins, since the other two methods were comparatively limited, and it also allows join conditions to be **clearly distinguished** from ordinary selection conditions within the WHERE clause.

3.9.1 A Complete Set of Relational Algebra Operations

Introduction

An important theoretical question in relational algebra is: which operations are truly **fundamental and necessary**, and which are merely included for **convenience**? This subsection addresses that question directly.

The Complete Set

It has been shown that the set of relational algebra operations $\{\sigma, \pi, \cup, \rho, -, \times\}$ — SELECT, PROJECT, UNION, RENAME, SET DIFFERENCE (MINUS), and CARTESIAN PRODUCT — forms a **complete set**. This means that **any other relational algebra operation** can be expressed as a **sequence of operations drawn from this set alone**.

INTERSECTION Is Not Strictly Necessary

Textbook Example: The INTERSECTION operation can be expressed entirely using UNION and MINUS:

$$R \cap S \equiv (R \cup S) - ((R - S) \cup (S - R))$$

Important Note: Although, strictly speaking, INTERSECTION is **not required** as a separate primitive operation (since it can always be derived from the complete set), it would be **highly inconvenient** to write out this complex expression every single time an intersection is actually needed — hence, INTERSECTION is retained as a **convenience operation**.

JOIN Is Not Strictly Necessary Either

Similarly, a JOIN operation can always be specified as a **CARTESIAN PRODUCT** followed by a **SELECT operation**, as discussed earlier:

$$R \bowtie_{\langle \text{condition} \rangle} S \equiv \sigma_{\langle \text{condition} \rangle} (R \times S)$$

In the same way, a **NATURAL JOIN** can be specified as a **CARTESIAN PRODUCT**, preceded by a **RENAME** operation and followed by **SELECT** and **PROJECT** operations. Hence, the various JOIN operations are also **not strictly necessary** for the full expressive power of relational algebra.

Why Convenience Operations Are Still Included

Despite not being strictly necessary from a purely theoretical standpoint, operations such as INTERSECTION and JOIN are considered important enough to include as **separate, named operations**, because they are **highly convenient to use** and are **very commonly applied** in real database applications. Several other operations are similarly included in the broader relational algebra **for convenience rather than strict necessity** — including the **DIVISION** operation, discussed in the next part.

Important Notes

- **EQUIJOIN** → JOIN using **only the = operator**; always leaves **redundant attribute pairs** with identical values.
- **NATURAL JOIN (*)** → EQUIJOIN **with the superfluous duplicate attribute removed**; requires **matching attribute names** (via RENAME if necessary).
- **Join selectivity** → **expected join result size** ÷ **maximum possible size** ($nR \times nS$).
- **No join condition** → JOIN **degenerates into CARTESIAN PRODUCT**.
- Standard JOINS are called **inner joins**, distinguished from **outer joins** (covered later).
- JOIN can also be applied across **more than two relations**, forming an **n-way join**.
- The **complete set** of relational algebra operations is $\{\sigma, \pi, \cup, \rho, -, \times\}$ — all other operations (INTERSECTION, JOIN, NATURAL JOIN, DIVISION) can be derived from this set, but are retained separately for **convenience**.

EQUIJOIN and NATURAL JOIN represent the most practically important specializations of the general JOIN operation, with NATURAL JOIN specifically refining EQUIJOIN by eliminating the redundant, duplicate attribute that equality-based joins always produce. While the theoretical foundation of relational algebra rests on just six core operations — SELECT, PROJECT, UNION, RENAME, MINUS, and CARTESIAN PRODUCT — from which every other operation, including INTERSECTION, JOIN, and NATURAL JOIN, can be formally derived, these additional operations are retained as first-class citizens of the algebra purely because of how frequently and conveniently they are used in real-world query formulation, avoiding the need to repeatedly write out their much longer equivalent expressions.

3.9.2 The DIVISION Operation

Introduction

The **DIVISION operation**, denoted by the symbol $\div$, is a specialized relational algebra operation that is useful for a **particular kind of query** that occurs fairly often in database applications — specifically, queries that ask for entities associated with **all** members of some other set, rather than just *some* of them. As noted in the previous part, DIVISION is **not strictly necessary** for the expressive power of relational algebra (it can be derived from the complete set $\{\sigma, \pi, \cup, \rho, -, \times\}$), but it is included as a **convenience operation** because it directly captures a query pattern that would otherwise require a fairly complex combination of other operations.

Motivating Example

Query: Retrieve the names of employees who work on **all** the projects that 'John Smith' works on.

This is the classic textbook example used to introduce DIVISION, and it is worked through step by step below.

Step-by-Step Construction — Textbook Example

Step 1 — Find the projects John Smith works on:

SMITH $\leftarrow \sigma \text{ Fname='John' AND Lname='Smith' (EMPLOYEE)}$

SMITH_PNOS $\leftarrow \pi \text{ Pno (WORKS_ON } \bowtie \text{ Essn=Ssn SMITH)}$

Explanation: First, the EMPLOYEE tuple for 'John Smith' is isolated in SMITH. Then, this is joined with WORKS_ON on the condition Essn = Ssn, and the result is projected over **Pno**, giving **SMITH_PNOS** — the complete list of project numbers that John Smith works on.

Step 2 — Build a relation of all (employee, project) work assignments:

SSN_PNOS $\leftarrow \pi \text{ Essn, Pno (WORKS_ON)}$

Explanation: This creates a relation containing a tuple $\langle \text{Essn, Pno} \rangle$ for **every** combination where the employee identified by Essn works on the project identified by Pno — essentially, the complete employee-to-project assignment relation drawn from WORKS_ON.

Step 3 — Apply DIVISION to find employees who work on ALL of Smith's projects:

SSNS(Ssn) $\leftarrow$ SSN_PNOS $\div$ SMITH_PNOS

RESULT $\leftarrow \pi$ Fname, Lname (SSNS * EMPLOYEE)

Explanation: Dividing SSN_PNOS by SMITH_PNOS produces exactly those Essn values (renamed to Ssn) that are associated, in SSN_PNOS, with **every single** project number appearing in SMITH_PNOS — that is, employees who work on **all** of John Smith's projects. The final RESULT then performs a NATURAL JOIN with EMPLOYEE to retrieve the actual first and last names of these qualifying employees.

General Definition of DIVISION

In general, the DIVISION operation is applied to two relations **R(Z) $\div$ S(X)**, where the attributes of S are a **subset** of the attributes of R — that is, **X $\subseteq$ Z**. Let **Y** be the set of attributes of R that are **not** attributes of S — that is, **Y = Z – X** (and therefore, **Z = X $\cup$ Y**).

The **result of DIVISION** is a relation **T(Y)** that includes a tuple **t** if:

- Tuples **tR** appear in R with **tR[Y] = t**, and
- **tR[X] = tS** for **every** tuple tS in S.

In plain terms: for a tuple t to appear in the result T of the DIVISION, the values in t must appear in R **in combination with every single tuple in S** — not just some of them, but all of them, without exception.

Important Note: In the formulation of DIVISION, the tuples in the **denominator relation S** act to **restrict the numerator relation R**, selecting only those tuples in the result that match **all** values present in the denominator. It is **not necessary to know in advance** what those specific values are, since they can themselves be computed using another operation — exactly as illustrated by the SMITH_PNOS relation in the example above, which was itself computed rather than hardcoded.

Illustrative Example — Abstract R $\div$ S

Consider a DIVISION operation where **X = {A}**, **Y = {B}**, and **Z = {A, B}**, applied to relations R and S:

Explanation: Suppose the values **b1** and **b4** appear in R **in combination with all three tuples present in S** — that is, for every value of A found in S, both b1 and b4 appear

paired with it in R. Because of this, **b1 and b4 appear in the resulting relation T**. In contrast, **all other values of B in R do NOT appear with every tuple in S**, and are therefore **not selected** — for instance, **b2 does not appear paired with a2**, and **b3 does not appear paired with a1**, so neither qualifies for inclusion in T.

Expressing DIVISION Using π , $\times$, and $-$

Since DIVISION is not a strictly primitive operation, it can be expressed as a **sequence of PROJECT, CARTESIAN PRODUCT, and SET DIFFERENCE** operations, as follows:

$$T1 \leftarrow \pi_Y(R) \quad T2 \leftarrow \pi_Y((S \times T1) - R) \quad T \leftarrow T1 - T2$$

Explanation: T1 first collects **all possible Y-values** that appear anywhere in R. T2 then identifies which of those Y-values, when combined with **every** tuple of S, produce a combination that is **missing** from R (i.e., disqualifying values). Finally, T is obtained by removing these disqualifying values (T2) from the full candidate set (T1), leaving only the Y-values that **do** appear with every tuple of S — precisely the definition of DIVISION.

Purpose and Relation to Universal Quantification

The DIVISION operation is defined purely **for convenience**, specifically to handle queries that involve **universal quantification** (see Section 8.6.7) — that is, queries phrased using an **"for all"** or **"all"** condition, such as "employees who work on **all** projects that Smith works on."

Important Note: Most RDBMS implementations, with **SQL as the primary query language**, **do NOT directly implement a DIVISION operator**. Instead, SQL handles this type of query using a **roundabout approach**, typically involving nested subqueries with NOT EXISTS or similar constructs (see queries Q3A and Q3B, Section 7.1.4).

Important Notes

- DIVISION ($\div$) captures queries involving **universal quantification** — "all," "every."
- $R(Z) \div S(X)$ requires $X \subseteq Z$; $Y = Z - X$ is the set of attributes appearing in the result.
- A tuple t appears in the result $T(Y)$ only if it is associated, in R, with **every single tuple** in S — not a partial match.

- **DIVISION can be expressed** using π , $\times$, and $-$ ($T1 \leftarrow \pi Y(R)$; $T2 \leftarrow \pi Y((S \times T1) - R)$; $T \leftarrow T1 - T2$), confirming it is **not a primitive operation**.
- **SQL does not implement DIVISION directly** — it is typically simulated using nested subqueries.
- The denominator relation **S restricts** the numerator relation **R**, without needing prior knowledge of S's specific contents — S can itself be the result of another query.

The DIVISION operation provides an elegant, purpose-built solution for queries that require **universal quantification** finding entities in one relation associated with **every** member of another relation a query pattern for which the standard SELECT, PROJECT, and JOIN operations alone would be cumbersome to express directly. Although DIVISION can be fully derived from the complete set of relational algebra operations using PROJECT, CARTESIAN PRODUCT, and SET DIFFERENCE, it is retained as a separate, **convenient operation** precisely because "all/every" queries, such as finding employees who work on all of John Smith's projects, occur frequently enough in practice to justify a **dedicated operator** — **even** though most real-world SQL implementations must simulate this behavior indirectly, since SQL itself has no direct DIVISION construct.

3.9.3 Notation for Query Trees

Introduction

Beyond writing relational algebra expressions as plain textual notation, RDBMSs internally require a **structured, visual way to represent queries** so that they can be processed, analyzed, and optimized efficiently. This section introduces the **query tree** notation — a tree-based data structure that corresponds directly to a relational algebra expression, and which plays an important role in how relational database management systems internally represent and execute queries.

Definition

A **query tree** — sometimes also called a **query evaluation tree** or **query execution tree** is a **tree data structure that corresponds to a relational algebra expression**. It includes the relational algebra operations being executed, and serves as a possible **data structure for the internal representation** of a query within an RDBMS.

Components of a Query Tree

A query tree consists of two main types of nodes:

- **Leaf nodes** — these represent the **input relations** of the query (the base tables being queried).
- **Internal nodes** — these represent the **relational algebra operations** being applied (such as SELECT, PROJECT, or JOIN) as the query is evaluated.

Execution of a Query Tree

The execution of a query tree proceeds according to the following rule: an **internal node's operation is executed whenever its operands** — represented by its **child nodes** — **become available**. Once an internal node's operation has been executed, that node is **replaced by the relation that results** from executing the operation. This process continues, moving progressively up the tree, until the **root node is executed**, at which point it produces the **final result relation** for the entire query.

Textbook Example — Query Tree for "Query 2"

Query: For every project located in 'Stafford', list the project number, the controlling department number, and the department manager's last name, address, and birth date.

This query, based on the COMPANY relational schema, corresponds to the following relational algebra expression:

π Pnumber, Dnum, Lname, Address, Bdate (((σ Plocation='Stafford' (PROJECT)) $\bowtie$ Dnum=Dnumber (DEPARTMENT)) $\bowtie$ Mgr_ssn=Ssn (EMPLOYEE))

Explanation of the Query Tree Structure:

- The **three leaf nodes**, labelled **P**, **D**, and **E**, represent the three base relations involved: **PROJECT**, **DEPARTMENT**, and **EMPLOYEE** respectively.
- The **relational algebra operations** in the expression — the SELECT, the two JOINS, and the final PROJECT — are represented as **internal tree nodes**.

Order of Execution Signified by the Query Tree

The query tree signifies an **explicit order of execution**. Specifically, in order to execute this query:

- The node marked **(1)** (the SELECT on PROJECT for Plocation='Stafford') **must begin execution before node (2)** (the JOIN with DEPARTMENT), because some

of the resulting tuples of operation (1) must already be available before operation (2) can begin.

- Similarly, **node (2) must begin executing and produce results before node (3)** (the JOIN with EMPLOYEE) can start execution, and so on, continuing up the tree toward the root.

Significance of the Query Tree

In general, a query tree provides a **good visual representation and understanding** of a query, expressed in terms of the specific relational algebra operations it uses. For this reason, it is **recommended as an additional means of expressing queries** in relational algebra, alongside the standard textual notation. Query trees will be revisited in greater depth when discussing **query processing and optimization**, since the structure and shape of a query tree directly influences how efficiently an RDBMS can evaluate a given query.

- A **query tree = leaf nodes (base relations) + internal nodes (relational algebra operations)**.
- Execution proceeds **bottom-up**: an internal node executes only once its child node(s) — its operands — are available.
- After execution, an internal node is **replaced by its resulting relation**, and this process continues until the **root produces the final query result**.
- The tree structure **explicitly encodes an order of execution** — operations lower in the tree must generally complete before operations higher up can begin.
- Query trees are used as an **internal representation** within RDBMSs, and are especially significant in the context of **query processing and optimization**.

The query tree offers a structured, visual counterpart to the textual relational algebra expression, representing base relations as leaf nodes and relational operations as internal nodes, with execution proceeding bottom-up from the leaves toward the root. The "Stafford project" example demonstrates how a query involving a SELECT followed by two successive JOINS and a final PROJECT can be naturally represented as a tree, with each internal node's execution depending on the availability of results from its child nodes below it. Beyond serving as a helpful visual aid for understanding a query's

structure, the query tree also forms the foundational internal data structure used by RDBMSs for query processing and optimization, a topic explored in greater depth in later chapters of the textbook.

3.10 – Additional Relational Operations

Introduction

The relational algebra operations discussed so far — SELECT, PROJECT, JOIN, UNION, and DIVISION — are powerful enough to express most common database queries. However, real-world commercial database applications frequently require operations that go beyond simple retrieval of stored attribute values. Businesses often need reports where the values displayed are not directly stored in the database but are *derived* from stored values through computation. For example, a payroll report may need to show the net salary of an employee, which is calculated as salary minus deductions, even though "net salary" is not itself a column stored in the EMPLOYEE table.

The basic PROJECT operation (π) only allows the user to select a subset of existing attributes from a relation; it does not allow any computation or transformation of those attribute values. To overcome this limitation, the relational algebra is extended with the **Generalized Projection** operation, which enhances the expressive power of the original relational algebra by allowing arithmetic expressions and functions to appear within the projection list itself.

This operation becomes especially important when developing reports, invoices, payslips, or any output where computed values must be produced as columns of the query result, rather than merely selected from existing columns.

Definition

The **Generalized Projection** operation extends the ordinary projection operation by allowing functions of attributes to be included in the projection list, in addition to just attribute names. It is formally denoted as:

$\pi F1, F2, \dots, Fn (R)$

Here, $F1, F2, \dots, Fn$ are functions defined over the attributes of relation R . These functions may involve arithmetic operations (such as addition, subtraction, multiplication, and division) as well as constant values combined with attribute values.

In simple terms, while ordinary projection can only "pick" columns, generalized projection can "compute" new columns using arithmetic expressions on the existing attribute values, and present the computed results as if they were columns of the output relation.

Need for Generalized Projection

The need for this operation arises because many practical database applications require derived or computed information rather than raw stored data. Examples include:

- Calculating net salary after deductions in a payroll system.
- Computing bonus amounts based on years of service.
- Calculating tax amounts as a percentage of salary.
- Generating invoice totals by multiplying quantity with unit price.

Since ordinary relational algebra projection cannot perform such arithmetic transformations, generalized projection was introduced specifically to handle these report-generation requirements, thereby making relational algebra more useful for real business applications.

Working of Generalized Projection

When generalized projection is applied to a relation R , each function F_i in the projection list is evaluated for every tuple of R . The functions may simply be attribute names (as in ordinary projection), or they may be arithmetic expressions combining one or more attributes with constants. The result is a new relation where each tuple contains the computed values of $F_1, F_2, \dots, F_n$ corresponding to one tuple of the original relation R .

Since the computed columns in the result do not have natural attribute names (they are expressions, not stored attributes), the RENAME operation (ρ) is commonly used together with generalized projection to assign meaningful names to the resulting columns. This combination allows the final report relation to have clean, readable column headings instead of raw expressions.

Textbook Example

Consider the relation:

EMPLOYEE (Ssn, Salary, Deduction, Years_service)

Suppose a report is required to display the following computed columns for each employee:

- **Net Salary** = Salary – Deduction
- **Bonus** = 2000 × Years_service
- **Tax** = 0.25 × Salary

Using generalized projection combined with renaming, this can be written as:

REPORT ← $\rho(\text{Ssn, Net_salary, Bonus, Tax}) (\pi \text{ Ssn, Salary – Deduction, } 2000 \times \text{Years_service, } 0.25 \times \text{Salary (EMPLOYEE)})$

Explanation: Here, the generalized projection first computes three new expressions — Salary minus Deduction, 2000 multiplied by Years_service, and 0.25 multiplied by Salary — for every tuple in EMPLOYEE, along with the Ssn attribute retained as is. The outer RENAME operation then assigns proper column names — Ssn, Net_salary, Bonus, and Tax — to the resulting relation REPORT, so that the final output is meaningful and easy to read, exactly as required in a payroll report.

- Generalized projection allows both attribute selection and arithmetic computation within a single projection list.
- The functions used may include constants, meaning fixed numeric values can be combined with attribute values (for example, $2000 \times \text{Years_service}$).
- RENAME is typically applied alongside generalized projection to give proper, business-meaningful names to computed attributes.
- Ordinary projection can be seen as a special case of generalized projection, where each function F_i is simply an attribute name with no computation involved.

Advantages

- Enables computation of derived values directly within the relational algebra, without needing external processing.
- Extremely useful for generating business reports such as payslips, invoices, and financial statements.

- Increases the expressive power of relational algebra beyond simple attribute retrieval.
- Can be combined with RENAME to produce clean, well-labelled output relations.

Limitations

- Generalized projection only supports arithmetic expressions and constants; it does not support complex procedural logic or conditional computations.
- The readability of the resulting relation depends on proper use of RENAME, since raw expressions as column headings are not meaningful to end users.

Generalized Projection is a vital extension to the basic relational algebra, introduced specifically to meet the needs of commercial reporting applications. By allowing arithmetic expressions and constants within the projection list, it enables the derivation of computed columns such as net salary, bonus, and tax directly from stored attribute values. Combined with the RENAME operation, generalized projection produces well-labelled, report-ready relations, making it an essential tool wherever raw stored data must be transformed into meaningful business information.

Chapter 8.4 – Additional Relational Operations

Introduction

The JOIN operations discussed earlier in Section 8.3 — including EQUIJOIN, NATURAL JOIN, and Theta JOIN — all share a common characteristic: they match tuples from two relations that satisfy a given join condition, and only those matching tuples appear in the final result. While this behaviour is correct and useful for many queries, it also means that any tuple which does not find a matching tuple in the other relation is **completely eliminated** from the JOIN result. This can lead to an undesirable loss of information whenever a user actually wants to see all the tuples from one or both relations, regardless of whether a match exists.

To address this limitation, relational algebra introduces a set of operations known as **OUTER JOINS**, which ensure that tuples without a match are still retained in the output, with missing values padded using NULLs, rather than being silently dropped from the result.

Need for Outer Join

Consider a real-world requirement: a company wants a report listing **all employee names**, along with the name of the department they manage, **if** they happen to manage one. If an employee does not manage any department, the report should still include their name, simply showing NULL in place of a department name — instead of omitting that employee altogether. This kind of "show everything, matched or not" requirement is exactly what motivated the development of outer join operations.

Inner Join vs Outer Join

The JOIN operations discussed earlier — where tuples with no match are eliminated — are collectively referred to as an **inner join**. In an inner join, such as a NATURAL JOIN operation $R * S$, only tuples from R that have matching tuples in S , and vice versa, appear in the result; tuples without a matching (or related) tuple, as well as tuples with NULL values in the join attributes, are **eliminated** entirely.

In contrast, **outer join** operations were developed for cases where the user wants to retain **all the tuples in R** , or **all those in S** , or **all those in both relations**, in the result of the JOIN — regardless of whether or not they have matching tuples in the other relation. This satisfies the requirement of combining tuples from two tables by matching corresponding rows, but without losing tuples that lack matching values.

LEFT OUTER JOIN

The **LEFT OUTER JOIN** operation, denoted by the symbol $\bowtie$, keeps **every tuple in the first (left) relation R** in the result of $R \bowtie S$. If a given tuple of R does not have a matching tuple in S , then the attributes of S in the joined result are filled, or "padded," with NULL values, while the tuple from R is still included in full.

Textbook Example:

TEMP $\leftarrow$ (**EMPLOYEE** $\bowtie$ Ssn=Mgr_ssn **DEPARTMENT**)

RESULT $\leftarrow$ π Fname, Minit, Lname, Dname (**TEMP**)

Explanation: Here, every employee tuple from EMPLOYEE is retained in the result, joined with DEPARTMENT wherever an employee's Ssn matches a department's Mgr_ssn (meaning that employee manages that department). For employees who do **not** manage any department, the Dname attribute in the result is padded with a NULL value instead of being dropped from the output. The final RESULT relation therefore lists every employee's name, along with the department they manage (or NULL if they manage none).

RIGHT OUTER JOIN

A similar operation, the **RIGHT OUTER JOIN**, denoted by the symbol $\bowtie$, keeps **every tuple in the second (right) relation S** in the result of $R \bowtie S$. If a tuple of S has no matching tuple in R, the attributes of R in the result are padded with NULL values, while the tuple from S is still fully included. This is essentially the mirror image of the LEFT OUTER JOIN, prioritising completeness of the right-hand relation instead of the left.

FULL OUTER JOIN

A third operation, the **FULL OUTER JOIN**, denoted by the symbol $\bowtie$, keeps **all tuples from both the left and right relations** in the result, even when no matching tuples are found on either side. Wherever a match does not exist, the missing attributes from the other relation are padded with NULL values as needed. This ensures that no tuple from either relation is lost, regardless of which side it belongs to.

Comparison Table

Operation	Symbol	Tuples Retained	Padding with NULL
INNER JOIN	$\bowtie$	Only matching tuples from both relations	None — unmatched tuples eliminated
LEFT OUTER JOIN	$\bowtie$	All tuples of left relation R	Right relation's attributes padded with NULL where unmatched
RIGHT OUTER JOIN	$\bowtie$	All tuples of right relation S	Left relation's attributes padded with NULL where unmatched
FULL OUTER JOIN	$\bowtie$	All tuples of both R and S	Attributes of whichever side lacks a match are padded with NULL

- Outer join operations were provided as an extension of relational algebra, developed in response to the typical business need to show related information from multiple tables **exhaustively**, meaning completely, without silent data loss.
- All three outer join operations — LEFT, RIGHT, and FULL — are part of the **SQL2 standard**.
- Outer joins are especially useful when a complete reporting of data from multiple tables is required, whether or not matching values exist between them.

Advantages

- Prevents unintentional loss of tuples that do not have a matching counterpart in the other relation.
- Supports realistic business reporting needs, such as listing all employees whether or not they manage a department, or all customers whether or not they have placed an order.
- Provides three flexible variants (LEFT, RIGHT, FULL) to suit whichever relation's completeness is required by the query.

Outer join operations extend the basic JOIN concept to overcome the information loss inherent in inner joins, where unmatched tuples are silently discarded. By introducing LEFT OUTER JOIN, RIGHT OUTER JOIN, and FULL OUTER JOIN, relational algebra allows queries to preserve all tuples of one or both relations, padding missing values with NULL wherever a match does not exist. This makes outer joins indispensable for real-world business applications that demand a complete and exhaustive view of related data across multiple tables, and their importance is reflected in their inclusion as a standard part of SQL2.

8.4 – Additional Relational Operations

OUTER UNION

Introduction

The ordinary UNION operation in relational algebra requires the two relations involved to be **union-compatible**, meaning they must have the same number of attributes, with corresponding attributes drawn from the same domain. However, in many practical situations, two relations may share only *some* of their attributes while differing in others, making them only **partially compatible**. Applying an ordinary UNION to such relations is not possible, since they fail the strict union-compatibility requirement.

To handle such situations, relational algebra introduces the **OUTER UNION** operation, which allows the union of tuples from two relations that are only partially union-compatible, by preserving all attributes from both relations and padding missing values with NULLs wherever necessary.

Definition

The **OUTER UNION** operation takes the union of tuples from two relations $R(X, Y)$ and $S(X, Z)$, where R and S are only **partially compatible** — meaning that only some of their attributes, represented as X , are union-compatible, while Y (unique to R) and Z (unique to S) are not shared between the two relations. The attributes that are union-compatible (X) are represented **only once in the result**, while the attributes that are not union-compatible from either relation (Y and Z) are also retained in the result relation, giving the final schema $T(X, Y, Z)$.

It is important to **note** that the OUTER UNION operation is essentially **the same as a FULL OUTER JOIN on the common attributes** — both preserve all tuples from both relations while combining shared and **unshared** attributes appropriately.

Partial Compatibility

Two relations are said to be partially compatible when they share **some**, but not all, of their attributes. In OUTER UNION, this shared portion (X) is used to match tuples between the two relations, similar to a join condition, while the remaining unshared attributes from each relation (Y and Z) are simply carried forward into the combined result, padded with NULLs where they do not apply.

Working

Two tuples t_1 in R and t_2 in S are said to **match** if $t_1[X] = t_2[X]$, meaning their values agree on all the shared attributes X . When such matching tuples are found, they are **combined (unioned) into a single tuple** in the result relation T , containing values for X , Y , and Z together. Tuples in either relation that have **no matching tuple** in the other

relation are still included in the result, but they are **padded with NULL values** for whichever attributes belong only to the other relation.

Textbook Example

Consider two relations:

STUDENT	(Name,	Ssn,	Department,	Advisor)
INSTRUCTOR	(Name, Ssn, Department, Rank)			

An OUTER UNION can be applied to these two relations, since they share the common attributes **Name, Ssn, and Department**. Tuples from the two relations are matched based on having the same combination of values for these shared attributes. The resulting relation is:

STUDENT_OR_INSTRUCTOR (Name, Ssn, Department, Advisor, Rank)

Explanation: All tuples from both STUDENT and INSTRUCTOR are included in the result, but tuples sharing the same (Name, Ssn, Department) combination appear only **once**. A tuple that appears only in STUDENT (a person who is a student but not an instructor) will have a NULL value for the Rank attribute, since Rank belongs only to INSTRUCTOR. Similarly, a tuple that appears only in INSTRUCTOR will have a NULL value for the Advisor attribute, since Advisor belongs only to STUDENT. A tuple representing a person who is **both** a student and an instructor — matching on Name, Ssn, and Department — will have values filled in for **all** its attributes, since both sides of the match contributed data.

Important Note — Same Person Appearing Twice

It is possible for the **same person to still appear twice** in the OUTER UNION result. For example, consider a graduate student in the Mathematics department who is also employed as an instructor in the Computer Science department. Although the two tuples representing this person in STUDENT and INSTRUCTOR will have the **same** Name and Ssn values, they will **not agree on the Department value** — one shows Mathematics (where they study) and the other shows Computer Science (where they teach). Since Department does not match between the two tuples, they will **not be combined**, and the person will appear as two separate tuples in the result.

This happens because the attribute **Department** carries two different meanings in the two relations: in STUDENT it represents the department where the person studies, while in INSTRUCTOR it represents the department where the person is employed. If the

OUTER UNION is intended to match purely on (Name, Ssn) regardless of department, the Department attribute in each relation should first be **renamed** to reflect its distinct meaning — for example, renaming it to **MajorDept** in STUDENT and **WorkDept** in INSTRUCTOR — and these renamed attributes should then be designated as **not** part of the union-compatible attribute set.

Difference between UNION and OUTER UNION

Point	UNION	OUTER UNION
Compatibility requirement	Relations must be fully union-compatible (same number and domain of attributes)	Relations need only be partially compatible (share some attributes)
Attributes in result	Same as the two input relations	Combination of shared attributes plus all unshared attributes from both relations
Handling of unshared attributes	Not applicable — schemas must already match	Unshared attributes retained, padded with NULL where they don't apply
Relationship to JOIN	Not related to JOIN	Equivalent to a FULL OUTER JOIN on the common attributes

Advantages

- Allows combining relations that are only partially compatible, which ordinary UNION cannot handle at all.
- Preserves all information from both relations, without losing tuples or attributes that are unique to either side.

- Useful in real-world scenarios where entities of similar but not identical types — such as students and instructors — need to be combined into a single consolidated view.

Limitations

- Careful attention must be paid to the meaning of shared attribute names; attributes with the same name but different real-world meaning (such as Department in STUDENT versus INSTRUCTOR) may cause incorrect matching unless properly renamed.
- The same real-world entity may still appear as multiple tuples in the result if the shared attribute values do not fully agree, which can complicate interpretation of the output.

Conclusion

The OUTER UNION operation extends the basic UNION operation to handle relations that are only partially union-compatible, allowing tuples from both relations to be combined while preserving attributes unique to each side, with NULLs used to pad missing values. Being equivalent to a FULL OUTER JOIN on the shared attributes, OUTER UNION is a valuable tool for consolidating related but non-identical relations, such as STUDENT and INSTRUCTOR, into a single unified relation — provided that attribute naming is handled carefully to avoid unintended mismatches.

VTU Quick Revision

Point	Detail
Applies to	Partially compatible relations (share only some attributes)
Shared attributes	Represented once in the result
Unshared attributes	Retained from both sides, padded with NULL where not applicable
Equivalent to	FULL OUTER JOIN on common attributes
Matching condition	$t1[X] = t2[X]$ on shared attributes X

Point	Detail
Caution	Rename same-named attributes with different meanings before applying OUTER UNION

8.5 Examples of Queries in Relational Algebra

Introduction

This section presents a series of **additional worked examples** to illustrate how the various relational algebra operations discussed so far — SELECT, PROJECT, JOIN, DIVISION, UNION, MINUS, INTERSECTION, and the AGGREGATE FUNCTION operation — are combined in practice to answer realistic database queries. All examples refer to the **COMPANY database** (the relational database state shown in Figure 5.6). An important point emphasized throughout this section is that **the same query can typically be stated in numerous different ways** using different combinations of relational algebra operations — each query below is presented in just **one** possible formulation, with the understanding that equivalent alternative formulations are also possible.

Query 1 — Employees Who Work for the 'Research' Department

Query: Retrieve the name and address of all employees who work for the 'Research' department.

```
RESEARCH_DEPT ← σ Dname='Research' (DEPARTMENT)
RESEARCH_EMPS ← (RESEARCH_DEPT ⋈ Dnumber=Dno EMPLOYEE)
RESULT ← π Fname, Lname, Address (RESEARCH_EMPS)
```

Explanation: First, **RESEARCH_DEPT** isolates the single DEPARTMENT tuple where Dname = 'Research'. Next, this is **joined** with EMPLOYEE on the condition Dnumber = Dno, giving **RESEARCH_EMPS** — all employee tuples belonging to that department. Finally, a **PROJECT** operation extracts only the Fname, Lname, and Address attributes.

As a single in-line expression:

π Fname, Lname, Address (σ Dname='Research' (DEPARTMENT) $\bowtie$ Dnumber=Dno (EMPLOYEE))

Note: This query could be formulated in other equivalent ways — for instance, the order of the JOIN and SELECT operations could be reversed, or the JOIN could be replaced with a **NATURAL JOIN** after first renaming one of the join attributes (Dno or Dnumber) to match the other.

Query 2 — Projects in 'Stafford' with Department Manager Details

Query: For every project located in 'Stafford', list the project number, the controlling department number, and the department manager's last name, address, and birth date.

STAFFORD_PROJS $\leftarrow \sigma$ Plocation='Stafford' (PROJECT)
 CONTR_DEPTS $\leftarrow$ (STAFFORD_PROJS $\bowtie$ Dnum=Dnumber DEPARTMENT)
 PROJ_DEPT_MGRS $\leftarrow$ (CONTR_DEPTS $\bowtie$ Mgr_ssn=Ssn EMPLOYEE)
 RESULT $\leftarrow \pi$ Pnumber, Dnum, Lname, Address, Bdate (PROJ_DEPT_MGRS)

Explanation: This query proceeds in a clear four-step chain: first, **select** the projects located in Stafford; then **join** them with their **controlling** departments (via Dnum = Dnumber); then **join** that result with the **department managers** (via Mgr_ssn = Ssn); and finally **project** onto the **specific attributes** required in the final answer. This is the same query illustrated earlier as a **query tree** example in Section 8.3.5.

Query 3 — Employees Who Work on ALL Projects Controlled by Department 5

Query: Find the names of employees who work on all the projects controlled by department number 5.

DEPT5_PROJS $\leftarrow \rho$ (Pno) (π Pnumber (σ Dnum=5 (PROJECT)))
 EMP_PROJ $\leftarrow \rho$ (Ssn, Pno) (π Essn, Pno (WORKS_ON))
 RESULT_EMP_SSNS $\leftarrow$ EMP_PROJ $\div$ DEPT5_PROJS
 RESULT $\leftarrow \pi$ Lname, Fname (RESULT_EMP_SSNS * EMPLOYEE)

Explanation: This is a classic **DIVISION** query. First, **DEPT5_PROJS** builds the checklist — the set of all project numbers controlled by department 5, renamed to **Pno**. Next, **EMP_PROJ** builds the full (Ssn, Pno) work-assignment relation from WORKS_ON, with attributes renamed to Ssn and Pno so they align correctly with DEPT5_PROJS for the DIVISION step. The **DIVISION** operation (EMP_PROJ $\div$ DEPT5_PROJS) then returns exactly those Ssn values associated with **every** project in

the checklist. Finally, a **NATURAL JOIN** with EMPLOYEE retrieves the actual Fname and Lname of these qualifying employees.

Important Note: The **renaming** of attributes (via ρ) in this query is essential — DIVISION requires the denominator relation's attributes to be a proper subset of the numerator relation's attributes, so both relations must be renamed consistently before the division can be correctly applied.

Query 4 — Projects Involving an Employee Named 'Smith' (Worker or Manager)

Query: Make a list of project numbers for projects that involve an employee whose last name is 'Smith', either as a worker or as a manager of the department that controls the project.

```
SMITHS(Essn) ←  $\pi$  Ssn ( $\sigma$  Lname='Smith' (EMPLOYEE))
SMITH_WORKER_PROJS ←  $\pi$  Pno (WORKS_ON * SMITHS)
MGRS ←  $\pi$  Lname, Dnumber (EMPLOYEE  $\bowtie$  Ssn=Mgr_ssn DEPARTMENT)
SMITH_MANAGED_DEPTS(Dnum) ←  $\pi$  Dnumber ( $\sigma$  Lname='Smith' (MGRS))
SMITH_MGR_PROJS(Pno) ←  $\pi$  Pnumber (SMITH_MANAGED_DEPTS * PROJECT)
RESULT ← (SMITH_WORKER_PROJS  $\cup$  SMITH_MGR_PROJS)
```

Explanation: This query is solved by handling its two "either/or" conditions separately, then combining them with **UNION**:

- **SMITH_WORKER_PROJS** finds project numbers for projects where a Smith is directly a **worker** (via WORKS_ON).
- **MGRS** first builds a relation of all department managers with their last names and department numbers. **SMITH_MANAGED_DEPTS** then filters this to departments managed by a Smith, and **SMITH_MGR_PROJS** finds the project numbers of all projects controlled by those Smith-managed departments.
- Finally, **UNION** combines both sets of project numbers — projects where Smith works directly, **or** projects controlled by a department Smith manages — with duplicates automatically eliminated.

As a single in-line expression:

$$\pi \text{ Pno } (\text{WORKS_ON} \bowtie \text{Essn=Ssn } (\pi \text{ Ssn } (\sigma \text{ Lname='Smith' } (\text{EMPLOYEE}))) \cup \pi \text{ Pno } ((\pi \text{ Dnumber } (\sigma \text{ Lname='Smith' } (\pi \text{ Lname, Dnumber } (\text{EMPLOYEE}) \bowtie \text{Ssn=Mgr_ssn DEPARTMENT}))) \bowtie \text{Dnumber=Dnum PROJECT})$$

Query 5 — Employees with Two or More Dependents

Query: List the names of all employees with two or more dependents.

Important Note: Strictly speaking, this query **cannot be expressed in the basic (original) relational algebra**. It requires the use of the **AGGREGATE FUNCTION operation (3)**, specifically with the **COUNT** aggregate function. It is assumed that dependents of the same employee have **distinct Dependent_name** values.

$$\begin{aligned} \text{T1}(\text{Ssn, No_of_dependents}) &\leftarrow \text{Essn } \mathfrak{Z} \text{ COUNT Dependent_name (DEPENDENT)} \\ \text{T2} &\leftarrow \sigma_{\text{No_of_dependents} > 2} (\text{T1}) \\ \text{RESULT} &\leftarrow \pi \text{ Lname, Fname } (\text{T2} * \text{EMPLOYEE}) \end{aligned}$$

Explanation: T1 groups DEPENDENT tuples by Essn and counts the number of distinct dependents for each employee, renaming the result attributes to Ssn and No_of_dependents. T2 then filters this down to only those employees with more than 2 dependents (i.e., two or more, matching the query's intent). Finally, a NATURAL JOIN with EMPLOYEE retrieves the corresponding names.

Query 6 — Employees with No Dependents

Query: Retrieve the names of employees who have no dependents.

Important Note: This is a classic example of a query that uses the **MINUS (SET DIFFERENCE)** operation.

$$\begin{aligned} \text{ALL_EMPS} &\leftarrow \pi_{\text{Ssn}} (\text{EMPLOYEE}) \\ \text{EMPS_WITH_DEPS}(\text{Ssn}) &\leftarrow \pi_{\text{Essn}} (\text{DEPENDENT}) \\ \text{EMPS_WITHOUT_DEPS} &\leftarrow (\text{ALL_EMPS} - \text{EMPS_WITH_DEPS}) \\ \text{RESULT} &\leftarrow \pi \text{ Lname, Fname } (\text{EMPS_WITHOUT_DEPS} * \text{EMPLOYEE}) \end{aligned}$$

Explanation: ALL_EMPS retrieves the Ssn of every employee in the company. EMPS_WITH_DEPS retrieves the Ssn (renamed from Essn) of every employee who has **at least one** dependent recorded in DEPENDENT. The **SET DIFFERENCE (MINUS)** operation then removes these "has dependents" Ssns from the full list of all employees, leaving exactly those employees with **no** dependents. A final **NATURAL JOIN** with EMPLOYEE retrieves their names.

As a single in-line expression:

$\pi \text{ Lname, Fname } ((\pi \text{ Ssn } (\text{EMPLOYEE}) - \rho \text{ Ssn } (\pi \text{ Essn } (\text{DEPENDENT}))) * \text{EMPLOYEE})$

Query 7 — Managers with At Least One Dependent

Query: List the names of managers who have at least one dependent.

$\text{MGRS}(\text{Ssn}) \leftarrow \pi \text{ Mgr_ssn } (\text{DEPARTMENT})$
 $\text{EMPS_WITH_DEPS}(\text{Ssn}) \leftarrow \pi \text{ Essn } (\text{DEPENDENT})$
 $\text{MGRS_WITH_DEPS} \leftarrow (\text{MGRS} \cap \text{EMPS_WITH_DEPS})$
 $\text{RESULT} \leftarrow \pi \text{ Lname, Fname } (\text{MGRS_WITH_DEPS} * \text{EMPLOYEE})$

Explanation: **MGRS** retrieves the **Ssn** (renamed from **Mgr_ssn**) of every employee who manages some department. **EMPS_WITH_DEPS** retrieves the **Ssn** of every employee who has at least one dependent. The **SET INTERSECTION** operation then finds exactly those **Ssns** present in **both** sets — that is, managers who **also** have at least one dependent. A final **NATURAL JOIN** with **EMPLOYEE** retrieves their names.

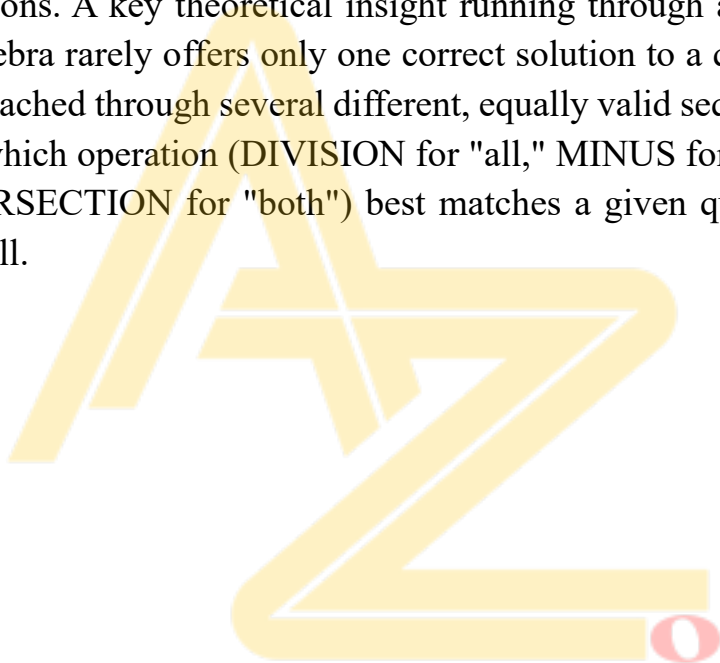
General Observation — Multiple Valid Formulations

As emphasized throughout this section, the **same query can be specified in many different ways** in relational algebra — the operations can often be applied in different **orders**, and in some cases, one operation can be **substituted for another** while still producing the same result. For example, the **INTERSECTION** operation used in Query 7 could equally well be replaced by a **NATURAL JOIN** between **MGRS** and **EMPS_WITH_DEPS**, producing the same set of qualifying **Ssns**.

- Query 1 illustrates a straightforward **SELECT + JOIN + PROJECT** pattern.
- Query 2 illustrates a **chained multi-JOIN** pattern (**SELECT** → **JOIN** → **JOIN** → **PROJECT**).
- Query 3 illustrates the **DIVISION** operation for "all/every" (universal quantification) queries, requiring careful **renaming** for compatibility.
- Query 4 illustrates combining **two separately derived project-number sets** using **UNION** for an "either/or" condition.
- Query 5 illustrates that some queries — specifically those involving counting or other aggregation — **cannot be expressed in basic relational algebra** and require the **AGGREGATE FUNCTION (3)** operation.

- Query 6 illustrates the **MINUS (SET DIFFERENCE)** operation for "has none of" style queries.
- Query 7 illustrates the **INTERSECTION** operation for "satisfies both conditions" style queries, and shows that INTERSECTION can often be substituted with an equivalent NATURAL JOIN.

These seven worked examples collectively demonstrate how the full toolkit of relational algebra operations SELECT, PROJECT, JOIN, NATURAL JOIN, DIVISION, UNION, MINUS, INTERSECTION, RENAME, and the AGGREGATE FUNCTION can be combined to answer a wide variety of realistic query patterns against the COMPANY database, ranging from simple attribute-based filtering to complex "all," "none," and "either/or" conditions. A key theoretical insight running through all these examples is that relational algebra rarely offers **only** one correct solution to a query the same result can typically be reached through **several** different, equally valid sequences of operations and recognizing which operation (DIVISION for "all," MINUS for "none," UNION for "either/or," INTERSECTION for "both") best matches a given query's phrasing is an essential exam skill.



Thank you

We're glad to be part of your engineering journey.
Keep exploring, keep innovating, and keep growing.

AZ Documents

Your Engineering Study Partner

www.azdocuments.in

Quality notes, simplified explanations, and
student-focused resources for every semester.

Connect With Us:

 www.azdocuments.in [[Click Here](#)]

 [@azdocuments](#) [[Click Here](#)]

 Join Our Student Community: [[Click Here](#)]

© 2025 AZ Documents. All rights reserved.

This material is intended for educational purposes only.
Redistribution without permission is prohibited.